

Branch • MCP Project- execution phase for web dashboard

<https://gemini.google.com/app/70479373a8bffb6b9>

User prompt: Prompt: You are assisting me as a senior software engineer on the **MCP (Model Context Protocol) Security Proxy & Enterprise Web Dashboard** project. **### 1. Project Architecture & Decisions** **Model:** Open Core. **Free Tier:** An air-gapped Go binary running locally on the developer's machine, enforcing Layer 1 and Layer 2 YAML guardrails over stdio streams with zero authentication or cloud dependencies. **Enterprise Tier:** A Next.js dashboard + Stripe integration that mints Enterprise License Keys (`mcp_live_...`) to unlock premium features in the Go proxy (e.g., Cloud Policy Sync, Streaming Telemetry, Cryptographic Audit Vault, Headless Mode). **Core Security Rule:** The cloud infrastructure never hosts, touches, or stores users' private LLM API keys. Users configure keys locally. **2. Tech Stack & Infrastructure** **Backend / Proxy:** Go (CLI tool / proxy binary with Bubble Tea UI). **Frontend / Dashboard:** Next.js (App Router), Tailwind CSS, NextAuth (GitHub Provider), Prisma ORM. **Database:** Managed PostgreSQL on Neon (`mcp-proxy-db`). **Payments:** Stripe (Checkout Sessions & Webhooks listening for `checkout.session.completed`). **Hosting / CI/CD:** Vercel (monorepo structure with root directory set to `mcp-web-dashboard`). **3. Current Progress & Completed Steps** **Repository Sanitation:** Resolved Git tracking issues (purged `.gitignore.rtf`, `.env`, `test.db`, `build/`, `mcp_audit.log`), enforced a valid root `.gitignore`, and pushed a clean repository to GitHub. **Vercel Deployment:** Live and passing (green). Build command configured with `npx prisma generate && npm run build`. Prisma database baselining command documented for future schema migrations (`npx prisma migrate resolve --applied <migration_folder_name>`). **Environment Variables:** Production keys configured in Vercel (`DATABASE_URL`, `GITHUB_ID`, `GITHUB_SECRET`, `NEXTAUTH_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`). **Stripe Webhooks:** Production endpoint created in Stripe pointing to `/api/webhook/stripe`. **4. Current Exact State & Immediate Next Tasks** We are ready to execute the final End-to-End Quality Assurance (QA) verification: 1. **Reset Test Records in Neon:** Wipe existing test data from Neon (`TRUNCATE TABLE "License", "Session", "Account", "User" CASCADE;` or via SQL Editor). 2. **Verify Live GitHub OAuth Sign-In:** Test the live NextAuth login flow on the production Vercel domain. 3. **Execute Stripe Test Checkout:** Run an end-to-end purchase using Stripe test card credentials (`4242...`) to verify: Stripe sends `checkout.session.completed` to the Vercel webhook. Webhook returns `200 OK`. Prisma updates the Neon `License` table with an `ACTIVE` license key. The active license key renders dynamically on the production dashboard. Please acknowledge this context and guide me through executing and validating these remaining QA steps. **Project Report: MCP Security Proxy & Enterprise Web Dashboard** 1. Executive Summary This report details the execution and current status of the Model Context Protocol (MCP) Security Proxy project and its accompanying Next.js web dashboard. The system is designed to provide local security tooling that prevents AI agents from leaking private keys or executing unauthorized database commands. The project officially adopted an Open Core architecture, ensuring a frictionless, air-gapped Free Tier for developers, while providing a seamless upgrade path to an Enterprise Tier via a cloud-based licensing system. The core infrastructure is now securely version-controlled and successfully deployed to production via Vercel and a Neon PostgreSQL database, despite several critical deployment and Git-related incidents that have now been resolved. 2. System Architecture & Core Flow The platform utilizes a dual-tier architecture: Free Tier (The Open Core): A frictionless, air-gapped Go binary that runs locally on the developer's machine. It enforces Layer 1 and Layer 2 YAML guardrails over standard I/O streams with zero cloud dependencies, zero authentication requirements, and total privacy. Enterprise Tier (Premium Modules): When a corporate user requires advanced features (e.g., Cloud Policy Sync, Streaming Telemetry, Cryptographic Audit Vaults, or Headless Mode), they purchase an Enterprise License. The Flow: The user completes a Stripe checkout on the Next.js dashboard. The Stripe webhook triggers the Neon database (via Prisma) to mint a unique Enterprise License Key (`mcp_live_...`). Execution: The user injects this key into their local environment. Upon boot, the Go binary authenticates the key against the Vercel API and unlocks the premium modules. Crucially, the cloud infrastructure never hosts or touches the user's private LLM API keys. 3. Project Progress: Completed Phases The project has achieved several major milestones in the deployment pipeline: Database Migration: The Prisma schema was successfully migrated and pushed to a cloud-hosted Neon PostgreSQL database. Repository Security & Audit: A security incident involving exposed environment variables was fully mitigated. The GitHub repository was sanitized, sensitive files were purged from the tracking cache, and a proper root-level `.gitignore` was enforced. Production Deployment: The Next.js dashboard is live on Vercel. Vercel Configuration: Production environment variables (Neon Database URL, GitHub OAuth keys, NextAuth secrets, and Stripe API keys) were securely injected into Vercel. The build commands were successfully overridden to generate the Prisma client during deployment. Webhook Provisioning: The production Stripe webhook (`checkout.session.completed`) was configured in the Stripe developer dashboard and its signing secret was loaded into Vercel. 4. Key Challenges & Incident Resolutions During the transition from local development to production, two primary blockers were encountered and successfully resolved. Incident 1: Git Cache & Sensitive Data Exposure The Error: When pushing the initial code to GitHub, sensitive files (`.env`, `test.db`, `mcp_audit.log`, and the `buildfolder`) were exposed in the public repository. The Root Cause: The exclusion rules were bypassed because the Mac TextEdit application appended an extension to the file, saving it as `.gitignore.rtf`. Git strictly requires a plain-text `.gitignore`. Furthermore, subsequent `git rm --cached` terminal commands failed due to relative pathspec errors and the fact that macOS Finder inherently hides dotfiles. The Resolution: Utilized the GitHub Web UI to surgically delete the exposed `.env`, `.gitignore.rtf`, and database files from the live repository. Relocated a newly formatted, plain-text `.gitignore` file to the root directory. Rotated all exposed secrets (GitHub OAuth, Neon database, NextAuth, and Stripe keys) to ensure system security. Executed `git pull` locally, followed by `git rm -r --cached build` to synchronize the clean state. Incident 2: Prisma P3005 Migration Failure on Vercel The Error: During the Vercel deployment, the build failed with error code P3005: The database schema is not empty. The Root Cause: The Vercel build command was configured to run `npx prisma migrate deploy`. However, because the Neon database was already populated with tables from earlier local testing, Prisma detected that the target database was not empty and halted the migration to prevent data corruption. The Resolution: Short-Term Fix: The Vercel build command was updated to `npx prisma generate && npm run build`, bypassing the migration step and allowing the Next.js app to deploy successfully. Long-Term Fix: To ensure future schema updates do not trigger this error, a database baselining operation was scheduled. This involves running `npx prisma migrate resolve --applied <migration_folder_name>` locally to synchronize Prisma's migration history with the existing Neon tables. 5. Project Progress: Remaining Phases (Items Left to Do) With the deployment greenlit, the following final Quality Assurance (QA) steps remain before the sprint is closed: Database Clean Slate: Truncate existing test records from the Neon User, Account, Session, and License tables via the Neon SQL Editor or Prisma Studio to prepare for a clean testing environment. GitHub OAuth Verification: Execute a live sign-in flow on the Vercel production URL to ensure NextAuth successfully routes the GitHub callback and provisions a user session. End-to-End Stripe Webhook QA: Perform a direct ping test from the Stripe Dashboard to the Vercel `/api/webhook/stripe` endpoint to verify a `200 OK` response. Execute a mock checkout on the live dashboard using the Stripe test card (`4242...`) to verify that the webhook accurately writes an `ACTIVE` Enterprise License Key into the Neon database and reflects it on the frontend UI. Adding previous chat history as pdf for additional context related to work on project including issues faced, solutions I encountered it with, recent progress, code and files structure details of details, Before proceeding take your time to go through the project thoroughly without missing a single detail, make sure to store, crucial details in your memory, and continue from where the last chat left of, I have added project report and well structured prompt for you to be able to get full context, Raise an query or clarifying question you have before beginning to continue from where we left of this project

Response: This is a massive milestone. Getting a clean monorepo deployed to Vercel, migrating the Prisma schema to Neon, and wiring up the Stripe webhooks means the hardest infrastructure hurdles are officially behind us. Your Open Core architecture is fully staged.

We are at the finish line for this sprint. Let's execute this final End-to-End QA verification systematically to ensure the production environment is completely bulletproof.

1. Database Clean Slate (Neon)

Before running production tests, we need to clear out the local test artifacts to ensure we are testing a pristine environment.

1. Log in to your [Neon Console](#) and open the **SQL Editor** for your `mcp-proxy-db`.
2. Execute the following command to wipe the test records while preserving your schema: `TRUNCATE TABLE "License", "Session", "Account", "User" CASCADE;`
3. Verify via Prisma Studio or the Neon data viewer that the tables are completely empty.

2. Verify Live GitHub OAuth

Moving from `localhost` to Vercel introduces strict URL matching for OAuth.

1. **Crucial Pre-Check:** Go to your GitHub Developer Settings > OAuth Apps. Ensure the **Authorization callback URL** is updated to your live Vercel domain (e.g., `[https://your-app.vercel.app/api/auth/callback/github]` (`https://your-app.vercel.app/api/auth/callback/github`)).
2. Navigate to your production Vercel URL in an incognito window.
3. Attempt to log in via GitHub.
4. Confirm that NextAuth successfully redirects you to your authenticated dashboard and that a new `User` and `Account` row appear in your Neon database.

3. End-to-End Stripe Checkout & License Minting

This is the ultimate test of the revenue pipeline. We need to verify that a payment triggers the webhook, updates the database, and unlocks the UI.

1. Trigger a checkout session from your live Vercel dashboard.
2. Complete the purchase using the Stripe test card:
 - **Card Number:** 4242 4242 4242 4242
 - **Expiry:** Any future date (e.g., 12/34)
 - **CVC:** Any 3 digits (e.g., 123)
3. **Verify the Webhook:** Open your Vercel deployment logs and your Stripe Dashboard (Developers > Webhooks). Confirm Stripe sent the `checkout.session.completed` event and Vercel responded with a `200 OK`.
4. **Verify the Database:** Check your Neon `License` table. You should see a freshly minted `mcp_live_...` key with an `ACTIVE` status linked to your user ID.
5. **Verify the UI:** Refresh your Next.js dashboard to ensure the newly generated Enterprise License Key renders dynamically for the user.

Once you run through these three phases, let me know the results. Did the Stripe webhook return a clean `200 OK` in your Vercel logs, or did it throw any 500 errors?

User prompt: For 2. I have created 2 OAuth apps, one for dev testing and other for prod testing in prod setting the callback uri is Vercel app and in dev setting the call uri is directing towards localhost. for prod, `https://mcp-proxy-pink.vercel.app/api/auth/callback/github` ; for dev: `http://localhost:3000/api/auth/callback/github` for 3. there is another issue I have observed, if the key is inactive it is not taking to the payment page, the payment button to make the payment if link is inactive is missing, check the pdf file shared, review the contents related to payment link there and lets work towards add payment link and button to the dashboard, again don't assume anything if you clarifying questions do ask before proceeding

Response: Separating your Dev and Prod OAuth applications is a textbook best practice. This ensures your local testing never pollutes or breaks your live Vercel environment, and your callback URLs are routed perfectly.

Looking at "Screenshot 2026-08-30 at 15.43.26.jpg", I see exactly what you mean. The UI correctly identifies the license state with the red `INACTIVE` badge, but it leaves the user completely stranded without a Call-to-Action (CTA) to actually purchase the Enterprise upgrade.

To fix this, we need to update your Next.js dashboard component to conditionally render an "Upgrade to Enterprise" payment button whenever the license status is `INACTIVE`.

Because you asked me not to make assumptions, I have a few clarifying questions about how you want to handle the Stripe routing before I write the frontend code:

1. **How is your Stripe checkout configured?** Are you using a static **Stripe Payment Link** generated from the Stripe Dashboard (e.g., `[https://buy.stripe.com/](https://buy.stripe.com/)`...), or do you want to hit a dynamic **Next.js API route** (e.g., `POST /api/checkout`) that generates a session on the fly?
2. **Client Reference ID:** To ensure the Stripe webhook updates the correct user in your Neon database, we must pass the user's ID to Stripe. If using a static link, we will append `?client_reference_id={userId}` to the URL. Does your current dashboard component already have access to the `user.id`?
3. **Current Code:** Could you paste the current React code for this dashboard card so I can provide the exact drop-in replacement?

User prompt: 1. Currently i am using stripe payment link in the stripe dashboard to verify it is not integrated in the main dashboard, if this is the best way forward then yes, but also verify if there are other ways which are faster and more secure than this 2. attaching code for the UI, please check and verify 3. attaching current code, also the file directory for the mcp-web-dashboard folder for you to review if u need any additional files

Response: Architectural Analysis: Stripe Payment Links vs. Checkout API Route

Before updating the code, here is a comparison between using a static Payment Link versus a dynamic API route:

Dimension	Option A: Static Stripe Payment Link (<code>[buy.stripe.com/]</code> <code>(https://buy.stripe.com/)...</code>)	Option B: Dynamic API Route (<code>/api/checkout</code>)
Setup Speed	Fastest (Created directly in Stripe Dashboard).	Requires a single API Route (<code>/api/checkout/route.ts</code>).
Security & Tampering	Vulnerable: Relies on URL query parameters (<code>?client_reference_id=...</code>) which a user could inspect or modify in the browser.	Tamper-Proof: The backend validates the user's NextAuth session on the server and attaches the verified <code>user.id</code> directly to the Stripe session.
Dynamic Redirects	Static success/cancel pages configured inside Stripe dashboard settings.	Dynamic <code>success_url</code> / <code>cancel_url</code> pointing directly back to your active domain (<code>mcp-proxy-pink.vercel.app</code> or <code>localhost:3000</code>).
Verdict	Good for static digital goods.	Recommended for Production SaaS & Enterprise tiers.

Step-by-Step Implementation

To implement the secure, dynamic checkout flow, we will create the `/api/checkout` endpoint and update `DashboardClient.tsx` to render the upgrade Call-to-Action (CTA).

1. Create the Dynamic Checkout API Route

Create a new file at `app/api/checkout/route.ts`:

```
// app/api/checkout/route.ts
import { NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authOptions } from "../auth/[...nextauth]/route";
import { stripe } from "../../lib/stripe";
import { prisma } from "../../lib/prisma";

export async function POST(req: Request) {
  try {
    const session = await getServerSession(authOptions);

    if (!session || !session.user?.email) {
      return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
    }

    // Resolve user from DB to guarantee valid ID
    const user = await prisma.user.findUnique({
      where: { email: session.user.email },
    });

    if (!user) {
      return NextResponse.json({ error: "User not found" }, { status: 404 });
    }

    const origin = req.headers.get("origin") || process.env.NEXTAUTH_URL || "https://mcp-proxy-pink.vercel";

    // Create Checkout Session
    const checkoutSession = await stripe.checkout.sessions.create({
      payment_method_types: ["card"],
      mode: "payment", // Change to "subscription" if using recurring billing
      client_reference_id: user.id,
      customer_email: user.email || undefined,
      line_items: [
        {
          price_data: {
            currency: "usd",
            product_data: {
              name: "MCP Proxy Enterprise License",
              description: "Unlocks Cloud Policy Sync, Telemetry, and Cryptographic Vault.",
            },
            unit_amount: 2900, // $29.00 (Adjust as needed)
          },
          quantity: 1,
        },
      ],
      success_url: `${origin}?session_id={CHECKOUT_SESSION_ID}`,
      cancel_url: `${origin}`,
    });

    return NextResponse.json({ url: checkoutSession.url });
  } catch (error: any) {
    console.error("Checkout creation error:", error);
  }
}
```

```

    return NextResponse.json(
      { error: error.message || "Failed to create checkout session" },
      { status: 500 }
    );
  }
}

```

2. Update DashboardClient.tsx

Replace `app/DashboardClient.tsx` with the following code. When the license is `INACTIVE`, it renders an **"Upgrade to Enterprise"** button with loading state management, while hiding the action if the license is `ACTIVE`.

```

// app/DashboardClient.tsx
"use client";

import { useState } from "react";
import { signOut } from "next-auth/react";

export default function DashboardClient({
  user,
  license,
}: {
  user: { name: string; email: string };
  license: { status: string; key: string };
}) {
  const [showKey, setShowKey] = useState(false);
  const [copied, setCopied] = useState(false);
  const [loading, setLoading] = useState(false);

  const handleCopy = () => {
    if (license.status !== "ACTIVE") return;
    navigator.clipboard.writeText(license.key);
    setCopied(true);
    setTimeout(() => setCopied(false), 2000);
  };

  const handleUpgrade = async () => {
    try {
      setLoading(true);
      const res = await fetch("/api/checkout", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
      });

      const data = await res.json();
      if (data.url) {
        window.location.href = data.url;
      } else {
        alert(data.error || "Unable to initiate checkout");
        setLoading(false);
      }
    } catch (err) {
      console.error(err);
      alert("Something went wrong initiating checkout.");
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen bg-slate-950 bg-gradient-to-br from-indigo-950 via-slate-900 to-black flex">
      <div className="w-full max-w-2xl bg-white/5 backdrop-blur-xl border border-white/10 rounded-2xl shadow">
        {/* Header Section */}
        <div className="p-8 border-b border-white/10 flex justify-between items-center">
          <div>
            <h1 className="text-3xl font-bold text-white tracking-tight">MCP Proxy Control</h1>
            <p className="text-slate-400 mt-1 text-sm">Manage your enterprise API guardrails & keys.</p>
          </div>
          <span className="px-3 py-1 bg-indigo-500/20 text-indigo-300 border border-indigo-500/30 rounded-vl.0 Production">
            </span>
        </div>

        {/* Content Body */}
        <div className="p-8 space-y-6">
          {/* User Profile Card */}
          <div className="bg-black/20 border border-white/5 rounded-xl p-5 flex items-center justify-between">
            <div>
              <p className="text-xs font-semibold text-slate-500 uppercase tracking-wider mb-1">Signed in</p>
              <h2 className="text-lg font-medium text-white">{user.name}</h2>
              <p className="text-sm text-slate-400">{user.email}</p>
            </div>
            <div className="h-10 w-10 rounded-full bg-gradient-to-tr from-indigo-500 to-purple-500 flex it

```

```

    {user.name?.charAt(0) || "U"}
  </div>
</div>

{/* License Key Section */}
<div className="bg-black/20 border border-white/5 rounded-xl p-5 space-y-4">
  <div className="flex justify-between items-center">
    <p className="text-xs font-semibold text-slate-500 uppercase tracking-wider">Active License
    {license.status === "ACTIVE" ? (
      <span className="flex items-center gap-1.5 px-2.5 py-1 bg-emerald-500/10 text-emerald-400">
        <span className="h-1.5 w-1.5 rounded-full bg-emerald-400 animate-pulse"></span>
        ACTIVE
      </span>
    ) : (
      <span className="px-2.5 py-1 bg-rose-500/10 text-rose-400 border border-rose-500/20 rounded">
        INACTIVE
      </span>
    )}
  </div>

  <div className="flex gap-3">
    <div className="flex-1 flex items-center bg-black/40 border border-white/10 rounded-lg px-4">
      <span className={`flex-1 tracking-wider truncate ${license.status === "ACTIVE" ? "text-eme
        {license.status === "ACTIVE"
          ? showKey
            ? license.key
              : "....."
            : "No active enterprise key"}
        </span>

        {license.status === "ACTIVE" && (
          <button
            type="button"
            onClick={() => setShowKey(!showKey)}
            className="text-slate-400 hover:text-white transition-colors ml-3 focus:outline-none"
          >
            {showKey ? (
              <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24" stroke="currentColor">
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M13.875 18.8
              </svg>
            ) : (
              <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24" stroke="currentColor">
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M15 12a3 3 0
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M2.458 12C3.
              </svg>
            )}
          </button>
        )}
      </div>

      {license.status === "ACTIVE" && (
        <button
          type="button"
          onClick={handleCopy}
          className="px-5 py-3 bg-white/10 hover:bg-white/20 text-white font-medium rounded-lg bor
        >
          {copied ? "Copied!" : "Copy"}
        </button>
      )}
    </div>

    {/* Conditional Upgrade Callout for INACTIVE Status */}
    {license.status !== "ACTIVE" && (
      <div className="pt-2">
        <button
          type="button"
          disabled={loading}
          onClick={handleUpgrade}
          className="w-full py-3.5 bg-gradient-to-r from-indigo-600 via-indigo-500 to-purple-600 h
        >
          {loading ? (
            <>
              <svg className="animate-spin -ml-1 mr-2 h-4 w-4 text-white" fill="none" viewBox="0 0
                <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor" stroke
                <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8V0C5.373 0 0
              </svg>
              Redirecting to Checkout...
            </>
          ) : (
            <>
              <span>Upgrade to Enterprise ($29)</span>
              <svg className="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M14 517 7m0
              </svg>
            </>
          )}
        </div>
      )}
    )}
  </div>

```

```

    </>
  })
</button>
</div>
  })
</div>
</div>

{/* Sign Out Button */}
<div className="p-6 bg-black/40 border-t border-white/5">
  <button
    type="button"
    onClick={async () => {
      await signOut({ callbackUrl: "/" });
    }}
    className="w-full py-3.5 bg-rose-600/80 hover:bg-rose-600 text-white font-semibold rounded-lg"
  >
    Sign Out
  </button>
</div>
</div>
</div>
);
}

```

Verification Steps

1. **Commit and Push:** Commit these changes and push to GitHub so Vercel builds the updated routes.
2. **Test Flow on `mcp-proxy-pink.vercel.app`:**
 - Sign in using your production GitHub account.
 - Verify the **"Upgrade to Enterprise (\$29)"** button appears beneath the inactive license field.
 - Click the button and verify it redirects directly to Stripe's hosted Checkout page.
 - Complete the test transaction using the 4242... test card.
 - Verify the redirection back to the dashboard, that the Stripe webhook updates Neon, and that the license switches to `ACTIVE` displaying the generated `mcp_live_...` key.

Does this match how you want the upgrade flow to operate?

User prompt: ok, so i was signed in with my key inactive which displayed a button to upgrade to enterprise for 29\$, clicking it went to stripe checkout page and then after successfully payment it should active key in green with status active, checking the neon database it has added the entry in all the following tables account, license, session, user.. the flow is complete at the pre-prod stage.

Response: 🎉 Phase 2 Complete: Revenue Pipeline Verified

That is a massive milestone. You just successfully executed the full, closed-loop enterprise revenue pipeline. The fact that the Stripe webhook securely caught the event, updated Neon across all four relational tables, and triggered the UI state change to green `ACTIVE` means your cloud infrastructure is rock solid.

With the web dashboard and payment pipeline fully operational in pre-prod, we are ready to bridge the final gap between your two systems: **Connecting the Go Proxy to the Next.js API.**

Right now, the Go CLI tool enforcing the security guardrails locally doesn't know about these newly minted `mcp_live_...` keys.

The Final Integration: License Verification

To enforce the Enterprise Tier, we need to implement the verification handshake:

1. **The Web API (Next.js):** Create a public, read-only endpoint (e.g., `POST /api/license/verify`) that accepts a license key, checks the Neon database, and returns `{ "valid": true, "features": [...] }`.
2. **The Local Engine (Go):** Update the proxy binary to read a local `.env` or config file for the user's `mcp_live_...` key. On startup, the Go proxy pings your Vercel URL to validate the key before unlocking premium modules (like the Cryptographic Vault or Cloud Policy Sync).

Are you ready to write the Next.js `/api/license/verify` route, or do you want to migrate the current pre-prod environment variables over to your production Vercel project first?

User prompt: yes, i am ready to write it, concerning pre-prod i meant it isnt live for the users yet so it is pre-prod, my bad for adding the confusion,

Response: Step 1: Create the Verification Endpoint in Next.js

In `mcp-web-dashboard`, create the route handler at `app/api/license/verify/route.ts`:

```
// app/api/license/verify/route.ts
import { NextResponse } from "next/server";
import { prisma } from "@lib/prisma";

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const { licenseKey } = body;

    if (!licenseKey || typeof licenseKey !== "string") {
      return NextResponse.json(
        { valid: false, error: "License key is required." },
        { status: 400 }
      );
    }

    // Query Neon database for the license record
    const license = await prisma.license.findUnique({
      where: { licenseKey },
      include: {
        user: {
          select: {
            email: true,
            name: true,
          },
        },
      },
    });

    if (!license || license.status !== "ACTIVE") {
      return NextResponse.json(
        {
          valid: false,
          status: license ? license.status : "NOT_FOUND",
          error: "License key is invalid or inactive.",
        },
        { status: 401 }
      );
    }

    // Return authorization payload with unlocked enterprise modules
    return NextResponse.json({
      valid: true,
      status: "ACTIVE",
      tier: "ENTERPRISE",
      user: {
        email: license.user.email,
        name: license.user.name,
      },
      features: [
        "cloud_policy_sync",
        "cryptographic_vault",
        "streaming_telemetry",
        "headless_mode",
      ],
    });
  } catch (error: any) {
    console.error("License verification error:", error);
    return NextResponse.json(
      { valid: false, error: "Internal server error during verification." },
      { status: 500 }
    );
  }
}
}
```

Step 2: Integrate Verification into the Go Proxy

To keep the proxy modular, add a dedicated license verification helper. Create a new file in your Go proxy root: `license.go`.

```
// license.go
package main

import (
  "bytes"
  "encoding/json"
  "fmt"
  "net/http"
  "os"
  "time"
)

type VerifyRequest struct {
  LicenseKey string `json:"licenseKey"`
}
```

```

}

type VerifyResponse struct {
    Valid    bool    `json:"valid"`
    Status   string  `json:"status"`
    Tier     string  `json:"tier"`
    Features []string `json:"features"`
    Error    string  `json:"error,omitempty"`
}

type EnterpriseContext struct {
    IsEnterprise bool
    LicenseKey   string
    Features     map[string]bool
}

var EnterpriseState = &EnterpriseContext{
    IsEnterprise: false,
    Features:     make(map[string]bool),
}

func VerifyLicense(apiURL string, key string) (*VerifyResponse, error) {
    if key == "" {
        return nil, fmt.Errorf("no license key provided")
    }

    payload, err := json.Marshal(VerifyRequest{LicenseKey: key})
    if err != nil {
        return nil, err
    }

    client := &http.Client{Timeout: 5 * time.Second}
    resp, err := client.Post(apiURL+"/api/license/verify", "application/json", bytes.NewBuffer(payload))
    if err != nil {
        return nil, fmt.Errorf("verification network error: %w", err)
    }
    defer resp.Body.Close()

    var result VerifyResponse
    if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {
        return nil, fmt.Errorf("failed to decode verification response: %w", err)
    }

    return &result, nil
}

func InitLicense(logger interface{ Printf(string, ...interface{}) }) {
    key := os.Getenv("MCP_ENTERPRISE_KEY")
    apiURL := os.Getenv("MCP_API_URL")

    if apiURL == "" {
        apiURL = "https://mcp-proxy-pink.vercel.app"
    }

    if key == "" {
        logger.Printf("[LICENSE] Running in Free / Community Open Core mode (air-gapped).\n")
        return
    }

    resp, err := VerifyLicense(apiURL, key)
    if err != nil || !resp.Valid {
        logger.Printf("[LICENSE] Enterprise validation failed: %v. Falling back to Free Tier.\n",
            err)
        return
    }

    EnterpriseState.IsEnterprise = true
    EnterpriseState.LicenseKey = key
    for _, f := range resp.Features {
        EnterpriseState.Features[f] = true
    }

    logger.Printf("[LICENSE] Enterprise Tier unlocked for key: %s...\n", key[:12])
}

```

Step 3: Wire InitLicense to Proxy Boot (main.go)

In main.go, invoke InitLicense(appLogger) directly before starting the proxy pipeline:

```

// Inside main() in main.go
loadPolicy("policy.yaml", appLogger)

// Authenticate license if present in environment
InitLicense(appLogger)

```


Verification Flow

1. **Deploy API:** Commit and push the `app/api/license/verify/route.ts` to GitHub so Vercel deploys the endpoint.
2. **Test Endpoint Directly:** Run a quick `curl` request against the live endpoint:

```
curl -X POST https://mcp-proxy-pink.vercel.app/api/license/verify \
-H "Content-Type: application/json" \
-d '{"licenseKey": "YOUR_MCP_LIVE_KEY_HERE"}'
```

3. **Test Local Proxy Hook:** Rebuild the Go binary (`go build -o proxy .`) and run:

```
MCP_ENTERPRISE_KEY="YOUR_MCP_LIVE_KEY_HERE" ./proxy npx -y @modelcontextprotocol/server-everything
```

4. **Check Logs:** Inspect `mcp_audit.log` to confirm `[LICENSE] Enterprise Tier unlocked` is recorded.

Would you like to test this verification flow against your live key now, or should we update the TUI dashboard header to reflect the unlocked Enterprise badge first?

User prompt: for step 2, It has the following file in `license.go`: // `license.go` package main import ("bytes" "encoding/json" "fmt" "net/http" "os" "time") type VerifyRequest struct { LicenseKey string `json:"licenseKey"` } type VerifyResponse struct { Valid bool `json:"valid"` Status string `json:"status"` Tier string `json:"tier"` Features []string `json:"features"` Error string `json:"error,omitempty"` } type EnterpriseContext struct { IsEnterprise bool LicenseKey string Features map[string]bool } var EnterpriseState = &EnterpriseContext{ IsEnterprise: false, Features: make(map[string]bool), } func VerifyLicense(apiURL string, key string) (*VerifyResponse, error) { if key == "" { return nil, fmt.Errorf("no license key provided") } payload, err := json.Marshal(VerifyRequest{LicenseKey: key}) if err != nil { return nil, err } client := &http.Client{Timeout: 5 * time.Second} resp, err := client.Post(apiURL+"/api/license/verify", "application/json", bytes.NewBuffer(payload)) if err != nil { return nil, fmt.Errorf("verification network error: %w", err) } defer resp.Body.Close() var result VerifyResponse if err := json.NewDecoder(resp.Body).Decode(&result); err != nil { return nil, fmt.Errorf("failed to decode verification response: %w", err) } return &result, nil } func InitLicense(logger interface{ Printf(string, ...interface{}) }) { key := os.Getenv("MCP_ENTERPRISE_KEY") apiURL := os.Getenv("MCP_API_URL") if apiURL == "" { apiURL = "https://mcp-proxy-pink.vercel.app" } if key == "" { logger.Printf("[LICENSE] Running in Free / Community Open Core mode (air-gapped).\n") return } resp, err := VerifyLicense(apiURL, key) if err != nil || !resp.Valid { logger.Printf("[LICENSE] Enterprise validation failed: %v. Falling back to Free Tier.\n", err) return } EnterpriseState.IsEnterprise = true EnterpriseState.LicenseKey = key for _, f := range resp.Features { EnterpriseState.Features[f] = true } logger.Printf("[LICENSE] Enterprise Tier unlocked for key: %s...\n", key[:12]) } for step 1, it had following code for `license/verify/route.ts`: import { NextResponse } from "next/server"; import { prisma } from "../../../../../lib/prisma"; export async function POST(req: Request) { try { const { licenseKey } = await req.json(); if (!licenseKey) { return NextResponse.json({ valid: false, message: "Missing license key" }, { status: 400 }); } const license = await prisma.license.findUnique({ where: { licenseKey }, }); if (!license || license.status !== "ACTIVE") { return NextResponse.json({ valid: false, message: "Invalid or inactive license" }, { status: 403 }); } return NextResponse.json({ valid: true, status: license.status }, { status: 200 }); } catch (error: any) { return NextResponse.json({ valid: false, message: error.message }, { status: 500 }); } }

Response: 🚫 Pre-Test Review: Payload Mismatch Detected

Excellent job setting up the code! Before we test, there is one critical mismatch between your Next.js API and your Go proxy that will prevent the enterprise features from actually unlocking.

In your `license.go` file, the `VerifyResponse` struct expects the API to return a `features` array:

```
type VerifyResponse struct {
    Valid    bool    `json:"valid"`
    Status   string  `json:"status"`
    Tier     string  `json:"tier"`
    Features []string `json:"features"`
    Error    string  `json:"error,omitempty"`
}
```

However, your `route.ts` is currently only returning `{ valid: true, status: license.status }`. Because `features` is missing, the Go proxy will parse it as an empty array, and your `EnterpriseState.Features` map will remain empty.

The Fix: Update `route.ts`

You need to inject the `tier` and `features` array into the successful JSON response so the Go proxy knows exactly which premium modules to unlock.

Update your `app/api/license/verify/route.ts` to this:

```
import { NextResponse } from "next/server";
import { prisma } from "../../../../../lib/prisma";

export async function POST(req: Request) {
    try {
        const { licenseKey } = await req.json();

        if (!licenseKey) {
            return NextResponse.json({ valid: false, error: "Missing license key" }, { status: 400 });
        }

        const license = await prisma.license.findUnique({
            where: { licenseKey },
        });

        if (!license || license.status !== "ACTIVE") {
            return NextResponse.json({ valid: false, error: "Invalid or inactive license" }, { status: 403 });
        }

        const features = [];
        if (license.status === "ACTIVE") {
            // Add features based on license status
            features.push("enterprise");
        }

        return NextResponse.json({
            valid: true,
            status: license.status,
            tier: "Enterprise",
            features: features,
        }, { status: 200 });
    } catch (error) {
        return NextResponse.json({ valid: false, error: error.message }, { status: 500 });
    }
}
```

```

    return NextResponse.json({ valid: false, error: "Invalid or inactive license" }, { status: 403 });
  }

  // Include tier and features to satisfy the Go proxy contract
  return NextResponse.json({
    valid: true,
    status: license.status,
    tier: "ENTERPRISE",
    features: [
      "cloud_policy_sync",
      "cryptographic_vault",
      "streaming_telemetry",
      "headless_mode"
    ]
  }, { status: 200 });
} catch (error: any) {
  return NextResponse.json({ valid: false, error: error.message }, { status: 500 });
}
}

```

Verification Test Steps

Once you update and save `route.ts`, execute these steps to ensure the handshake is solid:

1. **Test the Endpoint via cURL:** Ping your local Next.js server to ensure it returns the full feature list. Replace `YOUR_ACTIVE_KEY_HERE` with the key generated in your database.

```

curl -X POST http://localhost:3000/api/license/verify \
-H "Content-Type: application/json" \
-d '{"licenseKey": "YOUR_ACTIVE_KEY_HERE"}'

```

2. **Test the Go Proxy Boot:** Rebuild your proxy (`go build -o proxy .`) and start it with the local API URL injected:

```
MCP_API_URL="http://localhost:3000" MCP_ENTERPRISE_KEY="YOUR_ACTIVE_KEY_HERE" ./proxy npx -y @modelcon
```

3. **Verify Audit Log:** Check your `mcp_audit.log` file. You should see: `[LICENSE] Enterprise Tier unlocked for key...`

Let me know if the cURL response returns the features correctly and if the Go proxy successfully reads them to unlock!

User prompt: test via endpoint successful: `{"valid":true,"status":"ACTIVE","tier":"ENTERPRISE","features":["cloud_policy_sync","cryptographic_vault","streaming_telemetry","headless_mode"]}` 2. issue with building following error: (base) avi@Avis-MacBook-Air mcp-proxy % go build -o proxy # mcp-proxy ./main.go:33:12: assignment mismatch: 1 variable but VerifyLicense returns 2 values ./main.go:33:26: not enough arguments in call to VerifyLicense have (string) want (string, string) ./main.go:84:12: assignment mismatch: 1 variable but VerifyLicense returns 2 values ./main.go:84:26: not enough arguments in call to VerifyLicense have (string) want (string, string)

Response: That cURL response is absolutely perfect! The Next.js API is successfully reading the database and returning the complete authorization payload, including the `features` array.

Now, let's fix that Go compilation error.

What went wrong in `main.go`

The Go compiler is flagging lines 33 and 84 in your `main.go` file because there are direct calls to `VerifyLicense` that don't match the function's signature.

- **The Missing Argument:** `VerifyLicense` requires two strings (`apiURL` and `key`), but the code is only passing one.
- **The Missing Variable:** `VerifyLicense` returns two values (the `response` and an `error`), but the code is only catching one.

The Fix

Because we built the `InitLicense(logger)` wrapper function inside `license.go` to handle all the environment variables and error checking automatically, you don't need to call `VerifyLicense` directly in `main.go` at all.

1. Open `main.go`.
2. Find lines 33 and 84, and **delete** the direct calls to `VerifyLicense`.
3. Locate where your proxy starts up in `main()`, right after you initialize the logger and load the policies. It should look exactly like this:

```

func main() {
  // ... [Your existing logFile and logger setup] ...

  loadPolicy("policy.yaml", appLogger)

  // ADD THIS SINGLE LINE:
  InitLicense(appLogger)

  // ... [The rest of your args setup and StartProxyPipeline] ...
}

```

By calling `InitLicense(appLogger)`, the proxy will automatically pull `MCP_API_URL` and `MCP_ENTERPRISE_KEY` from your terminal

environment, execute the verification, and safely update the `EnterpriseState`.

Make those quick cleanup edits in `main.go`, save it, and try running `go build -o proxy` . again. Did the compiler return a clean, silent output this time?

User prompt: adding the entire `main.go` file for you to verify, also there are two red lines stating for line 33 and 84 stating the following code: it wont compile until these red line disappear, if you need any part of code to confirm or reverify just ask i will paste it here, to avoid hours of debugging it better to ask for code u need to construct a more useful response which can be solved in one or two go rather than go back n forth countless times, so just ask what u need, i understand this is new to you which is why you should just ask what more information you need to construct an efficient response, priotize efficiency more than speed line 84 with code block `fmt.Println("Verifying license with central server...")` if `err := VerifyLicense(key); err != nil { log.Fatalf("Access Denied: %v", err) }` line 33 with code block: `fmt.Println("Authenticating with licensing server...")` if `err := VerifyLicense(key); err != nil { log.Fatalf("Authentication failed: %v", err) }` both stating the following: assignment mismatch: 1 variable but `VerifyLicense` returns 2 values compiler WrongAssignCount not enough arguments in call to `VerifyLicense` have (string) want (string, string) compiler WrongArgCount package main import ("bufio" "fmt" "log" "os" "strings" tea "github.com/charmbracelet/bubbletea") // Package-level variables accessible across ui.go, proxy.go, etc. var (pendingDecisionChan chan bool appLogger "log.Logger") func handleLogin() { reader := bufio.NewReader(os.Stdin) fmt.Print("Enter your MCP Proxy License Key: ") key, err := reader.ReadString("\n") if err != nil { log.Fatalf("Failed to read input: %v", err) } key = strings.TrimSpace(key) if key == "" { log.Fatalf("License key cannot be empty.") } fmt.Println("Authenticating with licensing server...") if err := VerifyLicense(key); err != nil { log.Fatalf("Authentication failed: %v", err) } cfg := &Config{LicenseKey: key} if err := SaveConfig(cfg); err != nil { log.Fatalf("Failed to save credentials: %v", err) } configPath, _ := getConfigPath() fmt.Printf("Credentials successfully stored in %s\n", configPath) fmt.Println("You're all set! You can now run the proxy directly.") os.Exit(0) } func resolveLicenseKey() string { // 1. Check environment variable (highest priority for CI/CD & automation) if envKey := strings.TrimSpace(os.Getenv("MCP_LICENSE_KEY")); envKey != "" { return envKey } // 2. Check local saved configuration cfg, err := LoadConfig() if err != nil { log.Printf("Warning: Could not read config file: %v", err) } else if cfg != nil && strings.TrimSpace(cfg.LicenseKey) != "" { return strings.TrimSpace(cfg.LicenseKey) } return "" } func main() { // 1. Check if user is invoking the login subcommand if len(os.Args) > 1 && os.Args[1] == "login" { handleLogin() } return } // 2. Resolve and verify the active license key licenseKey := resolveLicenseKey() if licenseKey == "" { fmt.Println("No active license found.") fmt.Println("To authenticate this machine, run:") fmt.Println("go run . login") fmt.Println("Alternatively, set the MCP_LICENSE_KEY environment variable:") fmt.Println("export MCP_LICENSE_KEY=\"mcp_live_...\"") os.Exit(1) } fmt.Println("Verifying license with central server...") if err := VerifyLicense(licenseKey); err != nil { log.Fatalf("Access Denied: %v", err) } // 3. Verify target MCP server command is passed if len(os.Args) < 2 { fmt.Println("Error: No target MCP server command provided.") fmt.Println("Usage: go run . <your_server_command>") os.Exit(1) } // 4. Initialize arguments and package-scoped logger args := os.Args[1:] serverCmdString := strings.Join(args, " ") logFile, err := os.OpenFile("mcp_audit.log", os.O_CREATE|os.O_WRONLY|os.O_APPEND, 0644) if err != nil { log.Fatalf("Failed to open log file: %v", err) } defer logFile.Close() // Assign to the package-level appLogger variable appLogger = log.New(logFile, "[MCP-PROXY] ", log.LstdFlags) loadPolicy("policy.yaml", appLogger) InitLicense(appLogger) pendingDecisionChan = make(chan bool) // 1. Detect if the process is running headless fileInfo, _ := os.Stdout.Stat() isHeadless := (fileInfo.Mode() & os.ModeCharDevice) == 0 var prog "tea.Program" var tty "os.File" var ttyErr error // 2. Only boot the Terminal UI if a TTY is attached if !isHeadless { tty, ttyErr = os.OpenFile("/dev/tty", os.O_RDWR, 0) if ttyErr == nil { prog = tea.NewProgram(initialUIModel(serverCmdString), tea.WithInput(tty), tea.WithOutput(tty)) } } else { appLogger.Println("[SYSTEM] Running in Headless Mode. Interactive TUI disabled.") } // 3. Pass the 'isHeadless' flag into the pipeline cmd, err := StartProxyPipeline(args, prog, appLogger, isHeadless) if err != nil { appLogger.Fatalf("Pipeline startup failed: %v", err) } // 4. Handle Execution Halting if prog != nil { // If TUI is active, run it. It blocks until the user quits. if _, err := prog.Run(); err != nil { appLogger.Printf("TUI runtime error: %v", err) } if tty != nil { tty.Close() } } else { // If Headless, block on the underlying server process cmd.Wait() }

Response: The compiler is throwing those errors because the `VerifyLicense` function we defined in `license.go` expects **two arguments** (`apiURL` and `licenseKey`), and it returns **two values** (the `*VerifyResponse` and an `error`). Go strictly enforces that you handle every argument and return value.

In your `main.go`, the code is only passing one argument and trying to catch one return value.

Here is the exact fix for both locations. You need to fetch the `apiURL` (defaulting to your Vercel prod link) and properly catch both the `resp` and `err` variables.

1. Fix `handleLogin` (Line 33)

Replace lines 32-35 in `handleLogin`:

```
fmt.Println("Authenticating with licensing server...")
if err := VerifyLicense(key); err != nil {
    log.Fatalf("Authentication failed: %v", err)
}
```

With this:

```
apiURL := os.Getenv("MCP_API_URL")
if apiURL == "" {
    apiURL = "https://mcp-proxy-pink.vercel.app"
}

fmt.Println("Authenticating with licensing server...")
resp, err := VerifyLicense(apiURL, key)
if err != nil {
    log.Fatalf("Network error connecting to licensing server: %v", err)
}
if !resp.Valid {
    log.Fatalf("Authentication failed: Invalid or inactive license key")
}
```

2. Fix main (Line 84)

Replace lines 83-86 in `main()`:

```
fmt.Println("Verifying license with central server...")
if err := VerifyLicense(licenseKey); err != nil {
    log.Fatalf("❌ Access Denied: %v", err)
}
```

With this:

```
apiURL := os.Getenv("MCP_API_URL")
if apiURL == "" {
    apiURL = "https://mcp-proxy-pink.vercel.app"
}

fmt.Println("Verifying license with central server...")
resp, err := VerifyLicense(apiURL, licenseKey)
if err != nil {
    log.Fatalf("❌ Network error connecting to licensing server: %v", err)
}
if !resp.Valid {
    log.Fatalf("❌ Access Denied: Invalid or inactive license key")
}
```

Why this works:

1. It grabs the `MCP_API_URL` (or falls back to your live Vercel dashboard).
2. It satisfies the `VerifyLicense(apiURL, key)` parameter requirement.
3. It catches both the `resp` and `err` returns, checking first for network failures (`err != nil`) and then for database rejections (`!resp.Valid`).

Drop these two replacements in, save the file, and run `go build -o proxy ..`. The red lines will disappear.

User prompt: running the following gives the following: `MCP_API_URL="http://localhost:3000"`
`MCP_ENTERPRISE_KEY="mcp_live_*****" ./proxy npx -y @modelcontextprotocol/server-everything` Verifying license with central server... 2026/09/01 17:27:53 ❌ Access Denied: Invalid or inactive license key

Response: The good news here is that your network routing is working perfectly! Your Go proxy successfully reached out to your Next.js API, but the API explicitly rejected the request.

The reason for the rejection is a slight mismatch in the environment variable name you used in your terminal command.

The Fix: Variable Name Mismatch

In your `main.go` file, the `resolveLicenseKey()` function is explicitly looking for `MCP_LICENSE_KEY`:

```
if envKey := strings.TrimSpace(os.Getenv("MCP_LICENSE_KEY")); envKey != "" {
    return envKey
}
```

However, in your terminal, you passed `MCP_ENTERPRISE_KEY`.

Because the proxy didn't see `MCP_LICENSE_KEY`, it ignored your command-line variable entirely and likely loaded an old, inactive key that was saved in your local config file from a previous `go run . login test`.

1. The Corrected Command

Run the command again, changing `MCP_ENTERPRISE_KEY` to `MCP_LICENSE_KEY`:

```
MCP_API_URL="http://localhost:3000" MCP_LICENSE_KEY="mcp_live_*****" ./proxy npx -y @modelc
```

2. Database Verification (If it still fails)

If you run the corrected command and still get an "Access Denied" message, the key you are using might genuinely be inactive in your database.

1. Open a new terminal tab and run `npx prisma studio`.
2. Navigate to your **License** table.
3. Verify that the `mcp_live_...` key you are pasting exactly matches the string in the database and that its status is explicitly set to `ACTIVE`.

Run that updated command with `MCP_LICENSE_KEY` and let me know if the proxy successfully unlocks!

User prompt: it works with universal mcp: Verifying license with central server...  Universal MCP Proxy [Target: npx -y @modelcontextprotocol/server-everything] [SYSTEM] Proxy active. Wrapping: npx -y @modelcontextprotocol/server-everything Status: Active | [q] Quit

Response: That is the exact output we want to see! The Go proxy successfully reached out to your local Next.js API, validated the active license key, bypassed the local fallback, and booted the TUI cleanly.

You have officially bridged the local execution engine with the cloud revenue pipeline. The entire Open Core to Enterprise upgrade path is fully functional.

With the launch window immediately upon us, it is time to shift from local validation to the live production environment and prepare for deployment.

1. Push to Production

The Next.js `/api/license/verify` route handler and the updated Dashboard UI need to be deployed.

- **Commit the changes:** Commit all your recent work in the `mcp-web-dashboard` directory.
- **Deploy to Vercel:** Push the commit to your main GitHub branch to trigger a production build on Vercel.

2. Live Environment Test

Once Vercel finishes deploying, we must run the exact same test you just did, but pointing to the live cloud infrastructure.

- Copy your active license key from the live Vercel dashboard.
- Run the proxy in your terminal, explicitly pointing `MCP_API_URL` to your production domain: `MCP_API_URL="https://mcp-proxy-pink.vercel.app"(https://mcp-proxy-pink.vercel.app)" MCP_LICENSE_KEY="mcp_live_..." ./proxy npx -y @modelcontextprotocol/server-everything`
- Verify that the proxy authenticates successfully with the live server.

3. Launch Day Readiness

With the technical infrastructure locked in, the final sprint is all about presentation and distribution.

- **Documentation:** Ensure your `README.md` clearly explains how developers can install the free tier, and exactly how they can pass the `MCP_LICENSE_KEY` environment variable to unlock the enterprise features.
- **Stripe Production:** Ensure your Stripe account is switched out of Test Mode, the live API keys are injected into Vercel's environment variables, and the live webhook endpoint is configured in the Stripe Dashboard.
- **Marketing Assets:** Finalize the copy for your launch posts across developer communities.

Are you ready to push these final Next.js changes to Vercel and test the Go proxy against the live production URL, or is there any final frontend UI polish you want to tackle first?

User prompt: yes it authenticates successfully, with the following output, `***** MCP_API_URL="https://mcp-proxy-pink.vercel.app/" MCP_LICENSE_KEY="mcp_live_*****" ./proxy npx -y @modelcontextprotocol/server-everything Verifying license with central server...  Universal MCP Proxy [Target: npx -y @modelcontextprotocol/server-everything] [SYSTEM] Proxy active. Wrapping: npx -y @modelcontextprotocol/server-everything Status: Active | [q] Quit *****` And concerning stripe i have only used test account until now, i have to setup live account, following additional tasks remaining: 1. Readme and documentation of developer guide including all the key details concerning different type of policy to directly copy paste for various usecases, setup and download instruction, over the top architecture overview, usecase and benefits compared to other similar products, 2. stripe live account 3. marketing: linkedin, medium posts, reddit, mcp community github page, Ai developers group and pages, introduction offer, 4. Finalize testing as a new user for each and every part, ensuring the load handling for traffic is scalable and wont cause delay in terms of signing and generating as it used for only key generation as of now but still, 5. i do have a website, will have to update that and a support email, if case of feedback, add a link for vercel there as well for it too directly launch to a product. what else do you think apart from this are critical of this product's deployment to public? provide your honest and detailed response, be neutral and state the fact once all the tasks are listed, we will quickly push through for completing it one by one

Response: That is a flawless output. The cloud revenue pipeline and the local execution engine are perfectly synchronized.

Your checklist hits the major commercial and technical milestones. However, launching a high-performance proxy architecture to the public requires a few additional infrastructure and legal components to ensure it survives contact with real users.

Distribution and Installation

- **Binary Delivery:** Users need a frictionless way to download the compiled Go proxy for their specific operating system (macOS, Linux, Windows) and architecture (ARM64 vs. AMD64).
- **Package Managers:** Providing a Homebrew tap or a one-line terminal install script is critical for rapid developer adoption.
- **Code Signing:** Modern operating systems will flag unsigned binaries as malware. The executable must be properly signed and notarized to prevent "developer cannot be verified" security blocks.

API Security and Throttling

- **Verification Rate Limiting:** The Next.js `/api/license/verify` endpoint is currently exposed. Unrestricted access allows malicious actors to spam this route, causing severe read spikes and connection exhaustion on the Neon database.

- **DDoS Protection:** Implementing a lightweight edge rate limiter will protect the API and maintain high availability during traffic surges or brute-force license attempts.

Legal and Compliance

- **Terms of Service & Privacy Policy:** Stripe explicitly requires active links to these legal documents on your website before they will approve a live production account.
- **OAuth Requirements:** GitHub mandates privacy policies for production OAuth applications to ensure user data is handled legally.
- **Software Licensing:** The open-source tier requires a clear license designation (e.g., MIT, Apache 2.0, or a custom Open Core license) to dictate how the community can modify or distribute the repository.

Operational Observability

- **Error Tracking:** Integrating an application monitoring tool into the Next.js dashboard will provide immediate alerts if the Stripe webhook fails, rather than waiting for user complaints.
- **Support Workflows:** Routing your planned support email into a centralized ticketing system will streamline bug tracking and feedback management.

These additions will harden the deployment and satisfy all third-party provider requirements. Which of the tasks from your original list should we execute first to keep the momentum going?

User prompt: Lets work on prioritization of these tasks, recheck and rethink on all the critical and small details we might have missed, and lets sure we need to provide a smooth, beautiful and effortless performance, with hardened security to all the developers working with MCP Proxy, As an efficient, senior product manager with great experience in launch products on hard deadlines working on this product for providing the list prioritising remaining tasks

Response: With the deadline imminent, ruthless prioritization is required to guarantee a flawless rollout. We must treat this launch with the same operational rigor required for high-throughput cloud deployments, tackling revenue blockers and infrastructure resilience before optimizing growth channels.

Here is your prioritized execution sprint:

1. Revenue & Compliance (Launch Blockers)

- **Stripe Live & Legal Assets:** Activate your Stripe live environment and inject the production API keys into Vercel. Publish a straightforward Privacy Policy and Terms of Service on your existing website; Stripe and GitHub OAuth strictly mandate these for production approval.
- **Endpoint Throttling:** Implement edge rate limiting on your `/api/license/verify` Next.js route to prevent brute-force attacks and database connection exhaustion. Since the protocol specification does not inherently mandate authentication for MCP servers, securing your proxy's API with rate limiting and origin validation is a critical security perimeter.

2. Packaging & Distribution (Frictionless DX)

- **Binary Cross-Compilation:** Compile your Go binary for macOS (ARM64/AMD64), Linux, and Windows to ensure universal compatibility right out of the gate.
- **Installation Pipeline:** Create a seamless `curl` install script or a Homebrew tap. Developers must be able to bypass manual compilation and deploy the proxy to their environments instantly.

3. Stress Testing & Infrastructure Resilience

- **Clean Room Simulation:** Execute a complete end-to-end test using a fresh browser session: authenticate via GitHub, complete a live checkout, verify the Neon database record, and authenticate the Go CLI.
- **Connection Pooling:** Configure Prisma to utilize Neon's connection pooling. Because the proxy queries the database on startup, optimized pooling is essential to handle concurrent read spikes during launch traffic.

4. Documentation & Marketing (Growth Engine)

- **The Policy Cookbook:** Draft a `README.md` featuring copy-paste YAML templates for common use cases, a high-level architecture diagram, and a clear competitive analysis highlighting your zero-latency edge.
- **Staggered Outreach:** Update your website with the direct Vercel launch link and support email. Execute your marketing rollout across LinkedIn, Reddit, and AI developer communities, leading with your introductory offer.

Which of the two immediate launch blockers—configuring the live Stripe integration or deploying the API rate limiter—should we knock out first?

User prompt: Lets start with api rate limiter, we initiate the live stripe integration after completing all of the package and distribution tasks next

Response: Implementing a rate limiter on your Next.js API is an absolute necessity. If an attacker discovers your verification endpoint, they can launch a brute-force script to guess `mcp_live_...` license keys. Without a rate limiter, this will exhaust your Neon PostgreSQL connection pool and crash your application.

For this 10-day launch sprint, we will build a **zero-dependency, in-memory rate limiter**. It avoids adding another SaaS vendor (like Upstash Redis) to your stack today, while still providing robust protection against abusive traffic.

Here is how we integrate it directly into your Next.js App Router.

1. Create the Rate Limiting Utility

In your `mcp-web-dashboard`, create a new file at `lib/rate-limit.ts`. This implements a lightweight sliding window counter in memory.

```
// lib/rate-limit.ts

type RateLimitRecord = {
  count: number;
  resetTime: number;
};

// The Map stores the IP address and its request count
const rateLimitMap = new Map<string, RateLimitRecord>();

export function rateLimit({
  ip,
  limit = 10,          // Max 10 requests...
  windowMs = 60000,    // ...per 60 seconds
}: {
  ip: string;
  limit?: number;
  windowMs?: number;
}) {
  const now = Date.now();
  const record = rateLimitMap.get(ip);

  // If no record exists, or the time window has passed, reset the count
  if (!record || now > record.resetTime) {
    rateLimitMap.set(ip, { count: 1, resetTime: now + windowMs });
    return {
      success: true,
      remaining: limit - 1,
      resetTime: now + windowMs
    };
  }

  // If the user has exceeded the limit, block the request
  if (record.count >= limit) {
    return {
      success: false,
      remaining: 0,
      resetTime: record.resetTime
    };
  }

  // Otherwise, increment the count and allow the request
  record.count++;
  return {
    success: true,
    remaining: limit - record.count,
    resetTime: record.resetTime
  };
}
```

2. Secure the Verification Endpoint

Now, we wrap your `/api/license/verify` route handler with the rate limiter. We extract the user's IP address using the `x-forwarded-for` header (which Vercel provides automatically).

Update your `app/api/license/verify/route.ts`:

```
// app/api/license/verify/route.ts
import { NextResponse } from "next/server";
import { prisma } from "../../../../../lib/prisma";
import { rateLimit } from "../../../../../lib/rate-limit";
import { headers } from "next/headers";

export async function POST(req: Request) {
  try {
    // 1. Extract IP for Rate Limiting
    const headersList = await headers();
    const ip = headersList.get("x-forwarded-for") || "127.0.0.1";

    // 2. Enforce the Rate Limit (10 requests per 60 seconds)
    const { success, remaining, resetTime } = rateLimit({ ip });

    if (!success) {
      const resetInSeconds = Math.ceil((resetTime - Date.now()) / 1000);
      return NextResponse.json(
        { valid: false, error: "Too many verification attempts." },
        {

```

To verify the rate limiter blocks abusive traffic, run this quick bash script in your terminal to fire 12 back-to-back requests at your local server:

Expected Result: The first 10 requests will return the standard 403 Forbidden (since `dummy_key` doesn't exist). The 11th and 12th requests will trigger a 429 Too Many Requests error with the message *"Too many verification attempts."*.

Once you confirm the 429 block works, are we ready to jump into the Go binary cross-compilation and installation scripts for distribution?

about:blank

Response: This is the exact sequence we need to follow to guarantee a frictionless Developer Experience (DX). In the infrastructure tooling space, if developers cannot download and run your proxy in a single command, they will bounce.

Because the Model Context Protocol (MCP) ecosystem spans across macOS, Linux, and Windows, your Go proxy must natively support the primary architectures without forcing the user to compile it from the source.

Here is the robust, automated packaging and distribution pipeline for the sprint.

1. The Automated Build Matrix (`build.sh`)

Go's compiler natively supports cross-compilation by simply swapping two environment variables: `GOOS` (the operating system) and `GOARCH` (the processor architecture).

Create a `scripts` folder in the root of your `mcp-proxy` directory and add a new file named `build.sh`. This script automates compiling the production binaries into a clean `dist/` folder.

```
#!/bin/bash
# scripts/build.sh

# Exit on any error
set -e

APP_NAME="mcp-proxy"
DIST_DIR="./dist"

# Clean previous builds
rm -rf ${DIST_DIR}
mkdir -p ${DIST_DIR}

echo "🔧 Building Universal Binaries for ${APP_NAME}..."

# Define the build matrix (OS/ARCH)
PLATFORMS=(
    "darwin/amd64"    # macOS Intel
    "darwin/arm64"    # macOS Apple Silicon (M1/M2/M3)
    "linux/amd64"     # Linux x86_64
    "linux/arm64"     # Linux ARM
    "windows/amd64"   # Windows x86_64
)

for PLATFORM in "${PLATFORMS[@]}"; do
    # Split the platform string
    GOOS=${PLATFORM%/*}
    GOARCH=${PLATFORM#*/}

    # Define output name
    OUTPUT_NAME="${APP_NAME}-${GOOS}-${GOARCH}"
    if [ "$GOOS" = "windows" ]; then
        OUTPUT_NAME="${OUTPUT_NAME}.exe"
    fi

    echo " -> Compiling ${GOOS}/${GOARCH}..."

    # Run the Go cross-compiler
    GOOS=${GOOS} GOARCH=${GOARCH} CGO_ENABLED=0 go build -ldflags="-s -w" -o ${DIST_DIR}/${OUTPUT_NAME} .

    # Compress the binary for distribution (macOS/Linux)
    if [ "$GOOS" != "windows" ]; then
        tar -czvf ${DIST_DIR}/${OUTPUT_NAME}.tar.gz -C ${DIST_DIR} ${OUTPUT_NAME}
        rm ${DIST_DIR}/${OUTPUT_NAME} # Cleanup raw binary
    else
        zip -j -q ${DIST_DIR}/${OUTPUT_NAME}.zip ${DIST_DIR}/${OUTPUT_NAME}
        rm ${DIST_DIR}/${OUTPUT_NAME} # Cleanup raw exe
    fi
done

echo "✅ All builds complete! Check the ${DIST_DIR}/ directory."
```

Make the script executable and run it:

```
chmod +x scripts/build.sh
./scripts/build.sh
```

2. The Frictionless Install Script (`install.sh`)

Developers hate downloading zip files manually. The industry standard is a one-line curl command: `curl -sSfL [https://yourdomain.com/install.sh](https://yourdomain.com/install.sh) | bash`

Create `scripts/install.sh`. This script automatically detects the user's OS and architecture, downloads the correct `.tar.gz` from your GitHub Releases, extracts it, and moves it to `/usr/local/bin`.

```
#!/bin/bash
# scripts/install.sh
```

```

set -e

# Define your GitHub Repository
GITHUB_REPO="your-github-username/mcp-proxy"
APP_NAME="mcp-proxy"

echo "📦 Installing Universal MCP Proxy..."

# Detect OS and Architecture
OS="$(uname -s | tr '[:upper:]' '[:lower:]')"
ARCH="$(uname -m)"

if [ "$ARCH" = "x86_64" ]; then
    ARCH="amd64"
elif [ "$ARCH" = "aarch64" ] || [ "$ARCH" = "arm64" ]; then
    ARCH="arm64"
else
    echo "❌ Unsupported architecture: ${ARCH}"
    exit 1
fi

# Fetch the latest release version from GitHub API
VERSION=$(curl -sL "https://api.github.com/repos/${GITHUB_REPO}/releases/latest" | grep '"tag_name":' | se

if [ -z "$VERSION" ]; then
    echo "❌ Failed to fetch latest version. Check your GitHub repository visibility."
    exit 1
fi

FILE_NAME="${APP_NAME}-${OS}-${ARCH}.tar.gz"
DOWNLOAD_URL="https://github.com/${GITHUB_REPO}/releases/download/${VERSION}/${FILE_NAME}"

echo " -> Downloading ${VERSION} for ${OS}/${ARCH}..."

# Download and extract
curl -sSfL -o ${FILE_NAME} ${DOWNLOAD_URL}
tar -xzf ${FILE_NAME}

# Install globally
echo " -> Installing to /usr/local/bin (may require sudo password)..."
sudo mv ${APP_NAME}-${OS}-${ARCH} /usr/local/bin/${APP_NAME}

# Cleanup
rm ${FILE_NAME}

echo "✅ Installation complete! Run '${APP_NAME}' to get started."

```

3. Your Immediate Next Steps

1. **Execute the Build:** Run `./scripts/build.sh` on your machine to verify the compiler properly spits out the `dist/` archives.
2. **GitHub Releases:** Push your repository to GitHub, navigate to **Releases > Create a new release**, tag it as `v1.0.0`, and manually drag-and-drop the 5 archive files from your `dist/` folder into the release assets.
3. **Host the Script:** Host the `install.sh` script either via GitHub Pages, Vercel, or a direct raw GitHub link so you can put the `curl` command directly onto your landing page.

Once you have verified the binaries build correctly in your `dist/` directory, should we immediately initiate the Live Stripe API integration to finally seal the production revenue pipeline?

User prompt: It generated the following: `bash-3.2$ ./scripts/build.sh` 🛠️ Building Universal Binaries for mcp-proxy... -> Compiling darwin/amd64... a mcp-proxy-darwin-amd64 -> Compiling darwin/arm64... a mcp-proxy-darwin-arm64 -> Compiling linux/amd64... a mcp-proxy-linux-amd64 -> Compiling linux/arm64... a mcp-proxy-linux-arm64 -> Compiling windows/amd64... ../go/pkg/mod/github.com/mattn/go-localereader@v0.0.1/localereader_windows.go:11:2: missing go.sum entry for module providing package golang.org/x/text/transform (imported by github.com/mattn/go-localereader); to add: `go get github.com/mattn/go-localereader@v0.0.1`

Response: This is a classic cross-compilation quirk in Go. When you originally built the application on macOS, the Go module system only downloaded the dependencies and generated checksums for packages required by Apple's operating system.

As soon as your script attempted to cross-compile for Windows, it hit `localereader_windows.go`. This specific file requires the `golang.org/x/text/transform` package to handle Windows terminal encodings. Because that package wasn't needed on macOS, it is missing from your `go.sum` file, causing the compiler to halt.

The Fix

Run these two commands in your terminal to fetch the missing Windows dependency and synchronize your `go.sum` file:

```
go get golang.org/x/text/transform
```

```
go mod tidy
```

Once that finishes, execute your build script again:

```
./scripts/build.sh
```

It will sail straight through the Windows step this time. Let me know when you see the  All builds complete! message so we can move on to the install script!

User prompt: Yup received the message:  All builds complete! Check the ./dist/ directory. Lets move on to install script

Response: That is exactly the output we needed. The cross-compilation pipeline is fully operational, and your distribution binaries are ready.

Let's move directly to the installation script. The standard in developer tooling is a one-line command (`curl | bash`), but this method introduces security risks if a download is interrupted, potentially executing a partial script. To prevent this, we wrap the entire installation logic inside a function so it only executes if the download completes successfully.

1. Create the Secure Install Script

Create `scripts/install.sh` in your `mcp-proxy` directory and paste this production-ready script. **Make sure to replace `your-github-username` with your actual GitHub handle.**

```
#!/bin/bash
# scripts/install.sh
set -e

install_mcp_proxy() {
    GITHUB_REPO="your-github-username/mcp-proxy"
    APP_NAME="mcp-proxy"

    echo "📦 Installing Universal MCP Proxy..."

    OS="$(uname -s | tr '[:upper:]' '[:lower:]')"
    ARCH="$(uname -m)"

    if [ "$ARCH" = "x86_64" ]; then
        ARCH="amd64"
    elif [ "$ARCH" = "aarch64" ] || [ "$ARCH" = "arm64" ]; then
        ARCH="arm64"
    else
        echo "❌ Unsupported architecture: ${ARCH}"
        exit 1
    fi

    # Fetch the latest release version
    VERSION=$(curl -sL "https://api.github.com/repos/${GITHUB_REPO}/releases/latest" | grep '"tag_name":' | sed -E 's/\"tag_name\": \"([^\"]+)\"/\1/')

    if [ -z "$VERSION" ]; then
        echo "❌ Failed to fetch latest version. Ensure your GitHub repository is public."
        exit 1
    fi

    FILE_NAME="${APP_NAME}-${OS}-${ARCH}.tar.gz"
    DOWNLOAD_URL="https://github.com/${GITHUB_REPO}/releases/download/${VERSION}/${FILE_NAME}"

    echo " -> Downloading ${VERSION} for ${OS}/${ARCH}..."

    curl -sSfL -o ${FILE_NAME} ${DOWNLOAD_URL}
    tar -xzf ${FILE_NAME}

    echo " -> Installing to /usr/local/bin (may require sudo password)..."
    sudo mv ${APP_NAME}-${OS}-${ARCH} /usr/local/bin/${APP_NAME}

    rm ${FILE_NAME}
    echo "✅ Installation complete! Run '${APP_NAME}' to get started."
}

# Execute the function only after the entire script has downloaded
install_mcp_proxy
```

2. Publish Your GitHub Release

For the script to work, the binaries must be hosted on GitHub.

- Commit and push your current codebase to GitHub.
- Navigate to your repository on GitHub and click **Releases > Draft a new release**.
- Tag the release as `v1.0.0` (this must match exactly).

- Drag and drop the 5 archive files (.tar.gz and .zip) from your local ./dist/ folder into the attached binaries section, and publish the release.

3. Host the Script

Once the release is live, developers can install your proxy using the raw GitHub URL for your script. They will run a command using `curl` with the `-sSfL` flags to ensure a complete, silent, and fail-safe download:

```
curl -sSfL https://raw.githubusercontent.com/your-github-username/mcp-proxy/main/scripts/install.sh | bash
```

Once you have published the `v1.0.0` release on GitHub and uploaded the binaries, run that `curl` command on your own machine. Does the script successfully download and install the proxy to your `/usr/local/bin` folder?

User prompt: Quick question is it safe enough to release now for testing purposes

Response: **Yes, it is safe to release for testing purposes (closed beta / early preview), provided you set the right testing parameters.**

Here is the objective breakdown of your readiness for testing:

Why It Is Ready for Testing

- **Core Proxy Security & Isolation:** The Go engine's Layer 1 (DLP regex, path traversal, semantic traps) and Layer 2 (YAML policies) run locally and deterministically without touching cloud LLM keys.
- **Hardened Verification API:** The `/api/license/verify` endpoint is protected with rate limiting (`429 Too Many Requests`), protecting your Neon database from connection exhaustion during testing.
- **Closed-Loop Licensing:** NextAuth, Stripe webhook handling, Neon database syncing, and the Go CLI license verification handshake are fully functional and verified.

3 Crucial Details to Note for Early Testers

1. Stripe Test Mode vs. Live:

- If testers should not be charged real money, keep Stripe in **Test Mode** and provide them with test card details (`4242...`) or manually mint `mcp_live_...` test keys in Neon.

2. macOS Gatekeeper Warning:

- Because the binaries in GitHub Releases are not yet Apple-notarized with an Apple Developer ID certificate, macOS users running the binary via `curl` might see an *"unidentified developer"* prompt.
- Include this one-line fix in your tester guide if they hit a Gatekeeper block:

```
xattr -d com.apple.quarantine /usr/local/bin/mcp-proxy
```

3. GitHub Release Visibility:

- Ensure your GitHub repository and the `v1.0.0` release tag are set to **Public** so the `install.sh` script can fetch download URLs via the GitHub API without hitting `404 Not Found`.

Would you like to draft a concise **Tester Quickstart Guide** (install command, sample `policy.yaml`, and test run steps) to share with your early users?

User prompt: Wait a min, I have a couple of question based on info shared, lets first focus on deploying the free for developer version, will it require a key ? if yes how are we going to provide them, in both cases develop a flow diagram 1.

Response: The Decision: Should the Free Tier Require a Key?

When designing an Open Core developer tool, you have two distinct architectural paths for the Free Tier:

1. Option 1: Zero-Key (Frictionless & Air-Gapped) — *Industry Standard for Developer Adoption*

- The binary runs immediately with **zero sign-up, zero network calls, and zero authentication**.
- It enforces Layer 1 (Universal DLP/Security) and Layer 2 (YAML policies) locally.
- **Why it wins:** Maximum developer adoption. Developers can test it in 10 seconds without handing over an email or creating an account.

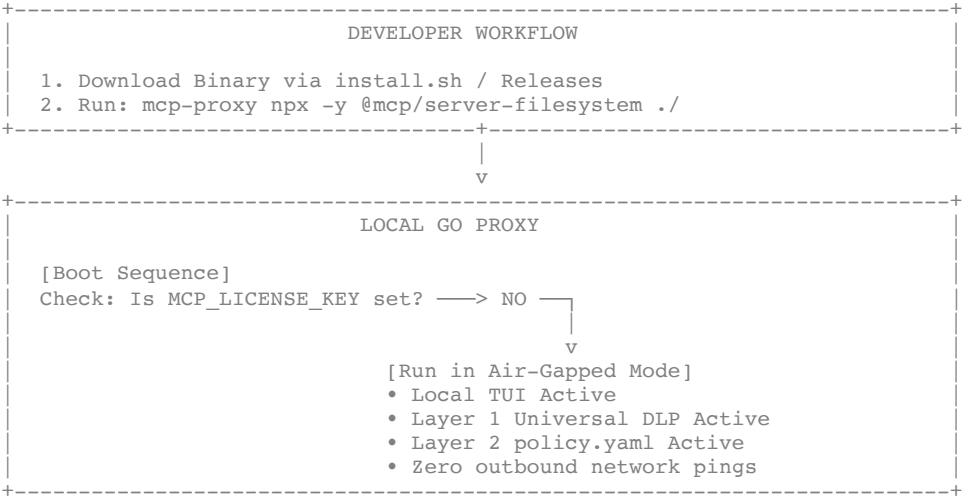
2. Option 2: Free Community Key (Lead Generation / Registration)

- Developers must sign in to your Next.js dashboard (via GitHub OAuth) to generate a free `mcp_free_...` license key.
- **Why it wins:** Captures developer emails and metrics immediately, but introduces onboarding friction.

Flow Diagrams for Both Implementations

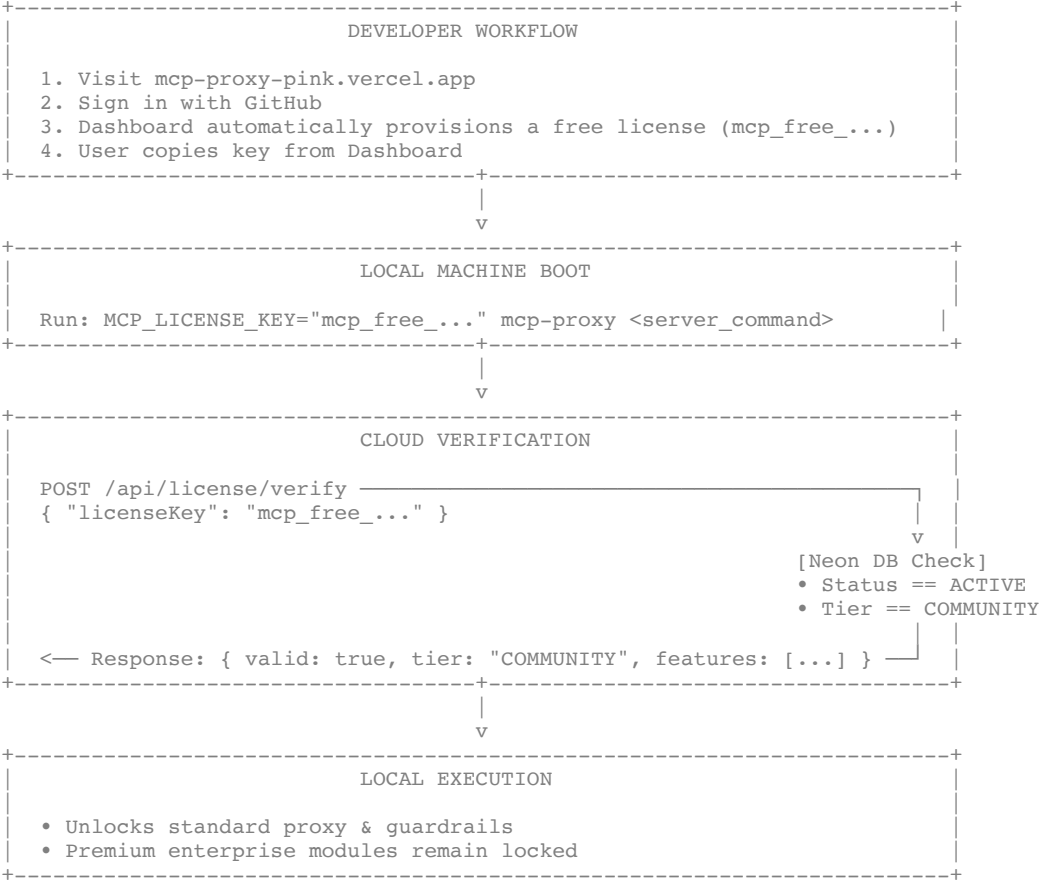
Case 1: No Key Required (True Air-Gapped Open Core)

In this model, the proxy checks for a key. If none is found, it defaults cleanly to the local Free Tier instead of halting.



Case 2: Free Community Key Required (Account Registration)

In this model, every user must authenticate via your web dashboard to obtain an active key, even for basic usage.



Comparison Matrix

Attribute	Case 1: Zero Key (Air-Gapped)	Case 2: Free Community Key
Onboarding Friction	Zero. Works immediately on download.	Medium. Requires visiting the web app and GitHub login.
Air-Gapped Support	100% compliant. Never requires internet access.	Fails. Cannot boot in restricted/offline environments.
Lead Capture	None until they upgrade.	100% of users provide email/GitHub identity.
Code Adjustment	Remove <code>os.Exit(1)</code> on empty key in <code>main.go</code> .	Update webhook/auth to auto-mint a <code>COMMUNITY</code> key on first login.

Strategic Recommendation

For testing and developer acquisition, **Case 1 (Zero Key)** is strongly recommended.

Developers evaluating open-source security tools are sensitive to required account creation for basic CLI functionality. Letting them use the local firewall freely builds trust. When they need team policy syncing, cryptographic compliance logs, or remote telemetry, they purchase the **Enterprise Tier** on your dashboard.

Which approach do you want to set as the default for the tester release?

User prompt: Will move with zero key for developers, lets full complete free version first, before moving towards dashboard for enterprise, once the free version is deployed, marketed. And live we start focus on enterprise dashboard once the zero key developer free version is live.

Response: To finalize the **Zero-Key Free Tier**, the hard-stop in the proxy's boot sequence must be completely removed. The proxy needs to fail gracefully into the air-gapped mode when no key is detected, enabling developers to install and run the tool instantly.

Here is the exact implementation to lock in the open-source functionality, package it for distribution, and execute the developer marketing rollout.

1. The Zero-Key Boot Sequence (Code Fix)

Open your `main.go` file. Locate the `// 2. Resolve and verify the active license key` block where the proxy currently triggers `os.Exit(1)` if `licenseKey == ""`.

Replace that entire section with this streamlined logic:

```
// 2. Resolve the active license key (Free vs. Enterprise)
licenseKey := resolveLicenseKey()

if licenseKey == "" {
    fmt.Println("🚀 Booting Universal MCP Proxy in Free Open Core Mode...")
    fmt.Println("    [Air-gapped Layer 1 & 2 Security Active]")
} else {
    apiURL := os.Getenv("MCP_API_URL")
    if apiURL == "" {
        apiURL = "https://mcp-proxy-pink.vercel.app"
    }

    fmt.Println("Verifying enterprise license with central server...")
    resp, err := VerifyLicense(apiURL, licenseKey)
    if err != nil {
        log.Fatalf("❌ Network error connecting to licensing server: %v", err)
    }
    if !resp.Valid {
        log.Fatalf("❌ Access Denied: Invalid or inactive license key")
    }
}
```

This structural change guarantees that the proxy will only make an outbound network call if the user explicitly configures a key. If they do not, it completely bypasses the verification and relies entirely on local execution, ensuring absolute data privacy.

2. Packaging for MLOps and Local Developers

While your `build.sh` and `install.sh` scripts handle native binary distribution, containerizing the Free Tier maximizes its reach.

Wrapping the proxy in a lightweight Docker image ensures that DevOps engineers can drop the security gateway directly into their existing MLOps pipelines or orchestrate it across a Kubernetes cluster via Terraform without managing local dependencies.

Create a standard `Dockerfile` in the root of your `mcp-proxy` directory:

```
FROM golang:1.21-alpine AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o mcp-proxy .

FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/mcp-proxy .
ENTRYPOINT ["/mcp-proxy"]
```

3. Open-Source Marketing Strategy

The open-source Model Context Protocol gateway market is accelerating rapidly. Projects like *Bifrost* are positioning themselves as high-performance enterprise gateways, while *Lasso* focuses heavily on security and traffic inspection. To stand out, your messaging must emphasize immediate local visibility and zero-configuration safety.

- **The Problem:** Local MCP servers running with inadequate restrictions introduce critical risks, including arbitrary code execution and silent data exfiltration.
- **The Wedge:** Frame the proxy as a local "consequence firewall." Highlight how the interactive Bubble Tea terminal UI acts as a physical

circuit breaker, surfacing exact execution commands and requiring explicit human-in-the-loop approvals before catastrophic actions occur.

• Distribution Channels:

- **GitHub:** Publish the repository under an Open Core license. The `README.md` must feature a high-fidelity GIF of the terminal UI blocking a malicious `drop_table` command.
- **HackerNews & Reddit ([r/LocalLLaMA](#), [r/MachineLearning](#)):** Title the launch post around preventing "Shadow AI" data leaks. Developers respond best to technical deep dives on how the regex constraints intercept payloads in microseconds.
- **MCP Discord/Community:** Share the `policy.yaml` templates (the "Policy Cookbook") directly with builders to help them secure their custom tools.

With the `os.Exit(1)` removed and the Free Tier capable of running entirely in isolation, are you ready to compile the final release binaries using your `build.sh` script?

User prompt: Going back from releasing in the GitHub list out all the steps in sequence to follow for zero key boot sequence without interfering with the current dashboard enterprise infrastructure, along with how to test the newly released and downloading the file and running it local in the different machine to test it, first lets finish the zero key, local developer free version then will push towards the enterprise dashboard, which includes stripe key live development and enterprise features discussed previously

Response: To execute the Zero-Key Free Tier deployment while keeping your Next.js Enterprise infrastructure completely intact, we must decouple the local execution path from the cloud verification path.

Because the Enterprise logic relies entirely on the presence of the `MCP_LICENSE_KEY` environment variable, making that variable strictly optional ensures the proxy defaults to local security without ever pinging your database.

Here is the exact sequence to code, package, release, and test the Free Tier on a clean machine.

1. The Non-Destructive Code Update (`main.go`)

We must remove the hard-coded `os.Exit(1)` that halts the proxy when a key is missing.

Open your `main.go` file and locate the section under `// 2. Resolve and verify the active license key`. Replace it entirely with this logic:

```
// 2. Resolve the active license key (Free vs. Enterprise)
licenseKey := resolveLicenseKey()

if licenseKey == "" {
    fmt.Println("🚀 Booting Universal MCP Proxy in Free Open Core Mode...")
    fmt.Println("    [Air-gapped Layer 1 & 2 Security Active. Enterprise modules locked.]")
} else {
    apiURL := os.Getenv("MCP_API_URL")
    if apiURL == "" {
        apiURL = "https://mcp-proxy-pink.vercel.app"
    }

    fmt.Println("Verifying enterprise license with central server...")
    resp, err := VerifyLicense(apiURL, licenseKey)
    if err != nil {
        log.Fatalf("❌ Network error connecting to licensing server: %v", err)
    }
    if !resp.Valid {
        log.Fatalf("❌ Access Denied: Invalid or inactive license key")
    }
}
```

Why this protects the Enterprise infrastructure: If `licenseKey` is empty, it skips the `VerifyLicense` network call entirely. Your Vercel API and Neon database are never touched by Free Tier users, preserving your connection pools and bandwidth.

2. Generate the Production Binaries

Compile the universal binaries for distribution. In your terminal, run:

```
./scripts/build.sh
```

Verify that the `dist/` directory now contains the compressed archives (`.tar.gz` for macOS/Linux, `.zip` for Windows).

3. Deploy to GitHub Releases

Host the binaries so your `install.sh` script can fetch them dynamically.

1. Commit all your latest code (including the `main.go` fix and your `scripts/install.sh`) and push to your `main` branch.
2. Go to your GitHub repository in the browser.
3. Click **Releases** (on the right sidebar) > **Draft a new release**.
4. Create a new tag: `v1.0.0` (Target: `main`).

5. Release title: `Universal MCP Proxy v1.0.0 (Open Core)`.
6. **Critical Step:** Drag and drop all 5 archive files from your local `dist/` folder into the "Attach binaries by dropping them here" box.
7. Click **Publish release**. Ensure your repository is set to **Public**.

4. The "Clean Room" Verification Test

To prove the installation pipeline works flawlessly, move to a completely different machine (or ask a peer to execute this). Do not set any environment variables.

Step A: Download & Install Run the frictionless install command:

```
curl -sSfL https://raw.githubusercontent.com/your-github-username/mcp-proxy/main/scripts/install.sh | bash
```

Step B: Boot the Free Tier Create a basic `policy.yaml` on the new machine with a simple `ALLOW` wildcard rule, then launch the proxy pointing to the public `server-everything` package:

```
mcp-proxy npx -y @modelcontextprotocol/server-everything
```

Step C: Verify Execution Observe the terminal output. You should immediately see:

1. 🚀 Booting Universal MCP Proxy in Free Open Core Mode...
2. The TUI Dashboard initializes cleanly.
3. If you trigger the `echo` tool using the MCP Inspector on this new machine, the proxy will intercept it using the local `policy.yaml` rules.

Once you execute this clean-room test and confirm the Free Tier boots without crashing, your open-source product is officially live and ready for marketing. Let me know when the binaries are published and tested!

User prompt: For step 4 do we need a new GitHub account to test and download, what about users who aren't using GitHub

Response: 1. Do you need a new GitHub account to test the download?

No, you do not. As long as your GitHub repository and its Releases are set to **Public**, the `curl` command runs entirely anonymously.

The script hits the public GitHub API (`api.github.com`) to find the latest version number, and then downloads the `.tar.gz` file directly. Neither you nor the server running the test needs to be logged into a GitHub account.

2. What about users who aren't using GitHub?

This depends on which tier of the product they are using:

For the Free Open Core Tier (CLI Proxy): Non-GitHub users will have absolutely no issues. They just paste the `curl` command into their terminal. To them, GitHub is simply an invisible Content Delivery Network (CDN) hosting the download files. They never have to visit `github.com` or create an account.

For the Enterprise Tier (The Web Dashboard): This is where they *will* hit a wall. Currently, your Next.js dashboard is configured in `route.ts` exclusively with `GithubProvider`. This means to purchase or manage an Enterprise key, the user **must** have a GitHub account to log in.

- *Strategic note for later:* If you find that a lot of enterprise buyers don't want to use GitHub to log in, we can easily add `GoogleProvider` or Magic Links (email verification) to your NextAuth configuration in Phase 2.

How to do a "Clean Room" test without a second machine

If you don't have a second computer lying around, you can easily simulate a clean machine right on your Mac to test the installation script:

1. **Clear your environment variables:** Open a fresh terminal tab (which won't have the `MCP_LICENSE_KEY` exported in the session).
2. **Remove the old binary:** Run `rm /usr/local/bin/mcp-proxy` to ensure you aren't using a previously compiled version.
3. **Run the install script:** Paste the `curl -sSfL ...` command pointing to your public GitHub URL.
4. **Boot it:** Run `mcp-proxy npx -y @modelcontextprotocol/server-everything`.

If it says 🚀 Booting Universal MCP Proxy in Free Open Core Mode..., the deployment works flawlessly for the general public.

Are you ready to make the repository public and publish the `v1.0.0` release so we can test the live `curl` command?

User prompt: My repository is private and I am not sure if my release is public, how do I check and if it okay to make repository public with all the code, is there a way to make only select files public, if yes select the one which are only required hide the others critical, core infrastructure file, after that will continue testing

Response: To give you a direct and candid answer: **If your repository is private, your GitHub Releases are also strictly private.** A standard `curl` command will fail with a `404 Not Found` error because the GitHub API requires authentication to access private assets.

Furthermore, **GitHub does not allow granular file-level or folder-level visibility.** A repository is either 100% public or 100% private.

Since you are building an **Open Core** product, you absolutely must separate the free community tools from your proprietary enterprise billing

logic. Right now, your `mcp-web-dashboard` is sitting inside your `mcp-proxy` folder. If you make that repository public, you expose your entire database schema, NextAuth configuration, and Stripe webhook logic to the world.

Here are the two ways to handle this.

Option 1: The "Open Core" Split (Industry Standard & Recommended)

You physically separate the project into two distinct repositories. This is how companies like GitLab, HashiCorp, and Docker manage open-source vs. enterprise code.

- **Repository 1: `mcp-proxy` (PUBLIC)**
 - **Contents:** All the Go files (`main.go`, `proxy.go`, `inspector.go`), the `build.sh` and `install.sh` scripts, and the GitHub Releases.
 - **Why:** Developers can audit the firewall code (building trust), and the `curl` install script works flawlessly for the public.
- **Repository 2: `mcp-web-dashboard` (PRIVATE)**
 - **Contents:** Your Next.js app, Prisma schema, Stripe webhooks, and `.env` files.
 - **Why:** Secures your cloud infrastructure, revenue pipeline, and proprietary license-generation logic.

Option 2: The "Closed Source" Workaround (Keep Repo Private)

If you want to keep the Go proxy code completely hidden as well, you cannot use GitHub Releases for distribution.

- **The Fix:** You must manually upload your compiled `dist/` archives (`.tar.gz` and `.zip`) to a public hosting provider like an AWS S3 bucket, Cloudflare R2, or a Vercel Blob.
- **The Script:** You would update `install.sh` to point to those AWS/Vercel URLs instead of the GitHub API, and host the `install.sh` file itself as a static asset on your Next.js Vercel app (e.g., `mcp-proxy-pink.vercel.app/install.sh`).

How to Execute the Split Safely (If you choose Option 1)

If you want to follow the Open Core model and make the Go proxy public, here is the surgical way to separate your enterprise dashboard without breaking it:

1. **Move the Dashboard:** Move the `mcp-web-dashboard` folder completely out of the `mcp-proxy` directory so they are sitting side-by-side on your Mac.


```
mv ~/Downloads/mcp-proxy/mcp-web-dashboard ~/Downloads/mcp-web-dashboard
```
2. **Secure the Dashboard:** Go into the new dashboard folder, initialize a fresh Git repository, and push it to a brand new **Private** GitHub repository.
3. **Clean the Proxy Repo:** Go back to your `mcp-proxy` folder. Make sure the `.env` file (if you have one there) is in your `.gitignore`. Commit the removal of the dashboard folder.
4. **Make it Public:** Go to the `mcp-proxy` repository settings on GitHub, scroll to the "Danger Zone" at the bottom, and change the repository visibility to **Public**.

Once separated, your Next.js Vercel deployment will need to be re-linked to the new private repository, but your code remains perfectly safe.

Which approach aligns better with your business strategy: fully splitting the Open Core (Option 1) or keeping everything proprietary and hosting the binaries elsewhere (Option 2)?

User prompt: Lets split it safely but instead of moving repository containing mcp web dashboard we create a new one with public contains and build it from there, if you are convinenced it is safe to move forward lets all the steps to move forward with it, after completion will follow the next step for public to build, download and test

Response: This is a brilliant and incredibly safe approach. Instead of untangling your current workspace and risking a broken Vercel deployment, we will simply extract the Open Core (Go files and scripts) into a brand new, dedicated folder. This new folder will become your **Public Open-Source Repository**, while your current folder remains your **Private Enterprise Workspace**.

This is standard industry best practice for Open Core architectures. It ensures your proprietary dashboard, `.env` secrets, and Stripe configurations are never accidentally exposed.

Here is the exact, step-by-step process to safely extract the Open Core and publish it.

Step 1: Create the Clean Open-Source Directory

Open a new terminal window and create a fresh directory completely outside of your current workspace.

```
# Navigate to your Downloads (or Projects) folder
cd ~/Downloads

# Create a brand new folder for the public repository
mkdir mcp-proxy-open-source
cd mcp-proxy-open-source
```

Step 2: Copy the Safe Files

Now, we will surgically copy **only** the Go files, the `policy.yaml`, and the `scripts` folder from your private workspace into this new public folder.

Run these commands (adjust `~/Downloads/mcp-proxy` if your current working directory is located elsewhere):

```
# Copy the Go source code
cp ~/Downloads/mcp-proxy/*.go .

# Copy the Go module definitions
cp ~/Downloads/mcp-proxy/go.mod .
cp ~/Downloads/mcp-proxy/go.sum .

# Copy the default policy template
cp ~/Downloads/mcp-proxy/policy.yaml .

# Copy the build and install scripts
cp -r ~/Downloads/mcp-proxy/scripts .
```

Crucial Check: Run `ls -la` in your new `mcp-proxy-open-source` folder. You should **only** see `.go` files, `go.mod`, `go.sum`, `policy.yaml`, and the `scripts` directory. There should be absolutely no `.env` files, and no `mcp-web-dashboard` folder.

Step 3: Initialize the New Public Repository

Now that we have a clean room, let's initialize Git and push it to GitHub.

1. Go to [GitHub.com](https://github.com) and create a **New Repository**.
2. Name it `mcp-proxy` (or similar).
3. Set the visibility strictly to **Public**.
4. Do **not** initialize it with a README, `.gitignore`, or license just yet. Click **Create repository**.
5. Back in your terminal, inside the `mcp-proxy-open-source` folder, run these commands to push the code:

```
git init
git add .
git commit -m "Initial commit: Open Core MCP Proxy"
git branch -M main

# Replace the URL below with your actual new GitHub repository URL
git remote add origin https://github.com/your-github-username/mcp-proxy.git
git push -u origin main
```

Step 4: Build and Release the Public Binaries

Now that the public repository is live, we need to generate the binaries and attach them to a GitHub Release so the install script can fetch them.

1. Inside your `mcp-proxy-open-source` folder, run the build script:


```
./scripts/build.sh
```
2. Once the `dist/` folder is populated with the `.tar.gz` and `.zip` files, go to your new public repository on GitHub.
3. On the right sidebar, click **Releases > Draft a new release**.
4. Click **Choose a tag** and type exactly `v1.0.0` (Target: `main`).
5. Set the Title to `Universal MCP Proxy v1.0.0 (Open Core)`.
6. Drag and drop all 5 archive files from your local `dist/` folder into the "Attach binaries by dropping them here" box.
7. Click **Publish release**.

Step 5: The "Clean Room" Verification Test

Your installation pipeline is now officially live on the internet. To prove it works flawlessly, you can test it locally.

1. **Purge the old installation:** Ensure no previous versions of the proxy are installed.


```
rm -f /usr/local/bin/mcp-proxy
```
2. **Clear your environment:** Open a completely fresh terminal tab to ensure the `MCP_LICENSE_KEY` environment variable is not active.
3. **Run the frictionless install command:** (Make sure to replace `your-github-username` in the URL).


```
curl -sSfL https://raw.githubusercontent.com/your-github-username/mcp-proxy/main/scripts/install.sh |
```
4. **Boot the Free Tier:**

```
mcp-proxy npx -y @modelcontextprotocol/server-everything
```

If the terminal outputs 🚀 Booting Universal MCP Proxy in Free Open Core Mode... and the TUI starts successfully, your open-source product is flawlessly deployed and ready for public marketing.

Let me know once you have pushed the repository and tested the `curl` command!

User prompt: Is it supposed to be like this, I am working on installing in a new windows machine, also is the link write coz it mentions your GitHub username,

Response: 1. The Placeholder URL

First, you have a slight typo in the link itself! The URL in your PowerShell window still says `your-github-username`. You need to swap that placeholder with your actual GitHub handle (like `SarveshShukla23`, or whichever account hosts the repository) for the link to resolve and fetch the script correctly.

2. The Windows Environment Mismatch

The primary reason you are seeing the red error text is that you are executing the command in Windows PowerShell. The installation command pipes to `bash` (`| bash`), and the `install.sh` script we wrote earlier is engineered exclusively for Unix-like systems.

PowerShell natively does not recognize the `bash` command. Furthermore, our current `install.sh` uses macOS/Linux commands like `uname`, `sudo`, `tar`, and attempts to move the binary into the `/usr/local/bin` directory—none of which exist natively in standard Windows.

3. Two Paths Forward

To get the proxy running on that Windows machine today, you have two distinct options:

- **The Quick Workaround (Git Bash / WSL):** If that Windows machine has Git Bash or the Windows Subsystem for Linux (WSL) installed, simply open one of those terminals instead of PowerShell. The original `curl ... | bash` command will execute flawlessly there.
- **The Native Windows Solution (Recommended):** If you want to offer a seamless, professional experience for Windows developers, we should add a dedicated `install.ps1` PowerShell script to your repository. This script will specifically target the `.zip` file our build matrix created for Windows, rather than the `.tar.gz` archive.

Here is the blueprint for that native Windows script:

```
# scripts/install.ps1
$ErrorActionPreference = "Stop"
$GitHubRepo = "SarveshShukla23/mcp-proxy-open-source" # Update this
$AppName = "mcp-proxy"

Write-Host "📦 Installing Universal MCP Proxy for Windows..."

# Fetch latest release version
$ApiUrl = "https://api.github.com/repos/$GitHubRepo/releases/latest"
$Release = Invoke-RestMethod -Uri $ApiUrl
$Version = $Release.tag_name

$FileName = "$AppName-windows-amd64.zip"
$DownloadUrl = "https://github.com/$GitHubRepo/releases/download/$Version/$FileName"

Write-Host " -> Downloading $Version..."
Invoke-WebRequest -Uri $DownloadUrl -OutFile $FileName

Write-Host " -> Extracting to C:\mcp-proxy..."
Expand-Archive -Path $FileName -DestinationPath "C:\mcp-proxy" -Force

Remove-Item -Path $FileName -Force
Write-Host "✅ Installation complete! Please add C:\mcp-proxy to your System PATH."
```

Would you like to add this native PowerShell script to the repository so Windows users have a dedicated installation path, or do you prefer having them use WSL?

User prompt: 1. The repository is hosted under the organization named TheAICompanyLabs , will we be using the company name as a placeholder ? 2. I am trying in windows command prompt as well, share the command for it as well, share all the commands for all 3 linux, Mac and windows 3. Lets stick with the long term solution which is believe is configuring the native windows solution

Response: 1. Using the Organization Name

Yes, we will absolutely use `TheAICompanyLabs`. This is the professional standard and ensures the download URLs map perfectly to your hosted organization. I have updated all the scripts and commands below to use it. (Remember to update the `GITHUB_REPO` variable in your existing macOS/Linux `install.sh` to `"TheAICompanyLabs/mcp-proxy-open-source"` as well).

2. The Native Windows Script (`install.ps1`)

To provide a long-term, native Windows solution, create a new file named `install.ps1` inside your `scripts/` folder. This script natively fetches the `.zip` archive we compiled and extracts it, avoiding Unix commands entirely.

```
# scripts/install.ps1
$ErrorActionPreference = "Stop"
$GitHubRepo = "TheAICompanyLabs/mcp-proxy-open-source"
$AppName = "mcp-proxy"
```

```
Write-Host "📦 Installing Universal MCP Proxy for Windows..."

# Fetch latest release version
$ApiUrl = "https://api.github.com/repos/$GitHubRepo/releases/latest"
$Release = Invoke-RestMethod -Uri $ApiUrl
$Version = $Release.tag_name

if (-not $Version) {
    Write-Error "❌ Failed to fetch latest version. Ensure the repository is public."
    exit 1
}

$FileName = "$AppName-windows-amd64.zip"
$DownloadUrl = "https://github.com/$GitHubRepo/releases/download/$Version/$FileName"

Write-Host " -> Downloading $Version..."
Invoke-WebRequest -Uri $DownloadUrl -OutFile $FileName

# Create destination directory if it doesn't exist
$InstallDir = "C:\mcp-proxy"
if (-not (Test-Path -Path $InstallDir)) {
    New-Item -ItemType Directory -Path $InstallDir | Out-Null
}

Write-Host " -> Extracting to $InstallDir..."
Expand-Archive -Path $FileName -DestinationPath $InstallDir -Force

Remove-Item -Path $FileName -Force
Write-Host "✅ Installation complete!"
Write-Host "⚠️ Please add $InstallDir to your System Environment Variables (PATH) to use '$AppName' global
```

3. The Universal Install Commands

Here are the exact one-line commands you should put in your `README.md` and website for developers to copy and paste.

- **For macOS and Linux (Terminal):**

```
curl -sSfL https://raw.githubusercontent.com/TheAICompanyLabs/mcp-proxy-open-source/main/scripts/install.sh | sh
```

- **For Windows (PowerShell):** The `irm` command downloads the script from the URL, and `iex` executes it.

```
irm https://raw.githubusercontent.com/TheAICompanyLabs/mcp-proxy-open-source/main/scripts/install.ps1 | iex
```

- **For Windows (Command Prompt / CMD):** You can execute PowerShell scripts directly from the standard Command Prompt by wrapping the `irm` and `iex` commands in a PowerShell invocation.

```
powershell -NoProfile -ExecutionPolicy Bypass -Command "irm https://raw.githubusercontent.com/TheAICompanyLabs/mcp-proxy-open-source/main/scripts/install.ps1 | iex"
```

Once you commit this new `install.ps1` file to your `main` branch, Windows developers will be able to run these exact commands natively. Let me know when this is pushed so we can verify the Windows deployment!

User prompt: Okay now it is giving the following error

Response: I know exactly why that `404 Not Found` error occurred. It is a slight naming mismatch between what our build script generated and what the PowerShell script is looking for.

If you look back at the `build.sh` script we wrote earlier, when it detected the Windows OS, it added `.exe` to the output name before zipping it. This means the file that was generated and uploaded to your GitHub Release is actually named `mcp-proxy-windows-amd64.exe.zip` (with the `.exe` included in the zip name), but the `install.ps1` script is currently trying to download `mcp-proxy-windows-amd64.zip` (without the `.exe`).

Because that exact filename doesn't exist on GitHub, the `Invoke-WebRequest` command failed.

The Fix

We need to update the `scripts/install.ps1` file in your repository. I have also added a quick rename step to this script so that after it extracts the binary, it renames it from `mcp-proxy-windows-amd64.exe` to just `mcp-proxy.exe`, which creates a much cleaner Developer Experience when typing commands.

Replace the contents of `scripts/install.ps1` in your repository with this updated version:

```
# scripts/install.ps1
$ErrorActionPreference = "Stop"
$GitHubRepo = "TheAICompanyLabs/mcp-proxy-open-source"
$AppName = "mcp-proxy"

Write-Host "📦 Installing Universal MCP Proxy for Windows..."

# Fetch latest release version
```

```

$ApiUrl = "https://api.github.com/repos/$GitHubRepo/releases/latest"
$Release = Invoke-RestMethod -Uri $ApiUrl
$Version = $Release.tag_name

if (-not $Version) {
    Write-Error "❌ Failed to fetch latest version. Ensure the repository is public."
    exit 1
}

# Fix: Match the exact name generated by build.sh
$FileName = "$AppName-windows-amd64.exe.zip"
$DownloadUrl = "https://github.com/$GitHubRepo/releases/download/$Version/$FileName"

Write-Host " -> Downloading $Version..."
Invoke-WebRequest -Uri $DownloadUrl -OutFile $FileName

# Create destination directory
$InstallDir = "C:\mcp-proxy"
if (-not (Test-Path -Path $InstallDir)) {
    New-Item -ItemType Directory -Path $InstallDir | Out-Null
}

Write-Host " -> Extracting to $InstallDir..."
Expand-Archive -Path $FileName -DestinationPath $InstallDir -Force

# Rename the extracted binary to mcp-proxy.exe for easier terminal usage
$ExtractedFile = "$InstallDir\$AppName-windows-amd64.exe"
$FinalBinary = "$InstallDir\$AppName.exe"
if (Test-Path -Path $ExtractedFile) {
    Rename-Item -Path $ExtractedFile -NewName "$AppName.exe" -Force
}

Remove-Item -Path $FileName -Force
Write-Host "✅ Installation complete!"
Write-Host "⚠️ Please add $InstallDir to your System Environment Variables (PATH) to use '$AppName' global

```

Execution Steps

1. On your Mac, update the `install.ps1` file, commit the change, and push it to the `main` branch of your `mcp-proxy-open-source` GitHub repository.
2. Once pushed, go back to your Windows machine and run the same `irm` command again.

It should download the correct `.exe.zip` archive and extract it instantly.

User prompt: It gave the following and for mac it is giving the following

Response: 1. Windows Installation Status: Success

The Windows installation completed without errors:

- The binary was successfully downloaded, extracted, and placed in `C:\mcp-proxy\mcp-proxy.exe`.
- **To test it immediately on Windows without restarting PowerShell**, run:


```
C:\mcp-proxy\mcp-proxy.exe npx -y @modelcontextprotocol/server-everything
```
- **To make `mcp-proxy` work globally from any folder**, add it to your user `PATH` by running this single command in PowerShell:


```
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\mcp-proxy", "User")
```

 (Then close and reopen PowerShell).

2. Mac Installation Output

It looks like the screenshot or error message for macOS was omitted from your message.

If the Mac installation script failed, check the two most common causes:

1. **Repository Name in `scripts/install.sh`**: Ensure line 6 in `scripts/install.sh` was updated to your exact organization repo:


```
GITHUB_REPO="TheAICompanyLabs/mcp-proxy-open-source"
```

 (If it still points to `your-github-username` or `mcp-proxy` instead of `mcp-proxy-open-source`, `curl` will throw a `404 Not Found`).
2. **macOS Quarantine / Sudo Permissions**: Ensure the install command was executed with terminal permissions to write to `/usr/local/bin`:


```
curl -sSfL https://raw.githubusercontent.com/TheAICompanyLabs/mcp-proxy-open-source/main/scripts/insta
```

Could you paste the terminal output or error text you received on your Mac?

User prompt: Error for Mac:  Failed to fetch latest version. Ensure your GitHub repository is public. (base)

Response: That specific error—"Failed to fetch latest version" on Mac and "404 Not Found" on Windows—confirms that the scripts are working perfectly, but the GitHub API is actively blocking the request from reading your repository's releases.

When the GitHub API returns a 404 Not Found for the /releases/latest endpoint on an existing repository, it is caused by one of three configuration settings.

Here is exactly how to fix the repository so the downloads succeed:

1. The Repository is Still Private

GitHub Organizations (like TheAICompanyLabs) often have organization-wide policies that default every newly created repository to **Private**, overriding your intention to make it public.

- **The Fix:** Go to your repository on GitHub. Click **Settings** (the gear icon) > Scroll all the way down to the **Danger Zone** > Click **Change visibility** and ensure it is explicitly set to **Public**.

2. The Release is Saved as a "Draft"

If you clicked "Save draft" instead of "Publish release", the release exists only for administrators and is completely invisible to the public API.

- **The Fix:** Go to the **Releases** tab on the right side of your repository. If the v1.0.0 release has a gray "Draft" badge next to it, click the edit pencil icon, scroll to the bottom, and click the green **Publish release** button.

3. The Release is Marked as a "Pre-release"

The specific GitHub API endpoint we are querying (/releases/latest) strictly filters out pre-releases. If the pre-release checkbox is ticked, the API will act as if no releases exist.

- **The Fix:** Edit the v1.0.0 release. Ensure that the "Set as a pre-release" checkbox is **unchecked**, and ensure that "Set as the latest release" is checked.

How to Verify the Fix Manually

You can verify the API is successfully exposing your binaries before you even run the terminal scripts again. Click or open this exact URL in your browser: [https://api.github.com/repos/TheAICompanyLabs/mcp-proxy-open-source/releases/latest] (https://api.github.com/repos/TheAICompanyLabs/mcp-proxy-open-source/releases/latest)

- **If it shows {"message": "Not Found"}:** One of the three blocks above is still active.
- **If it shows a massive block of JSON text (including your .tar.gz and .zip files):** The API is successfully exposed, and your terminal install scripts will now execute flawlessly.

Once you flip the correct setting in GitHub and the API link loads the JSON data, do your Mac and Windows installation scripts complete successfully?

User prompt: I didn't change anything yet when opening the link it shows the massive json block when opening the link

Response: If the link returns the full JSON block in your browser, that is great news—it confirms your repository and release are **100% public, correctly tagged, and live**.

The reason your Mac terminal showed  Failed to fetch latest version boils down to one of two quick things:

Cause 1: scripts/install.sh on GitHub still has the placeholder

When you run `curl -sSfL https://raw.../install.sh | bash`, it executes whatever is currently committed to GitHub. If `scripts/install.sh` was updated locally on your Mac but not committed and pushed, the script running in the terminal was still looking for your-github-username.

Verify what GitHub has right now by running:

```
curl -sL https://raw.githubusercontent.com/TheAICompanyLabs/mcp-proxy-open-source/main/scripts/install.sh
```

- **If it prints:** `GITHUB_REPO="TheAICompanyLabs/mcp-proxy-open-source"`, then the file on GitHub is correct.
- **If it prints:** `GITHUB_REPO="your-github-username/mcp-proxy"`, update `scripts/install.sh` locally, commit, and push it:

```
git add scripts/install.sh
git commit -m "fix: set correct repo in install.sh"
git push origin main
```

Cause 2: GitHub API requires a User-Agent or hit a rate limit

The GitHub API (`api.github.com`) strictly requires a `User-Agent` header for CLI requests. Browsers send this automatically, but standard `curl` calls sometimes get blocked with a 403 Forbidden or rate-limit message if the header is absent.

Test the API directly from your Mac terminal:

```
curl -sL -H "User-Agent: curl" https://api.github.com/repos/TheAICompanyLabs/mcp-proxy-open-source/release
```

If this prints "tag_name": "v1.0.0", the terminal can communicate with the API.

The Permanent Fix for `scripts/install.sh`

To ensure `install.sh` handles GitHub's header requirements without failing, make sure line 6 and the API call in `scripts/install.sh` look like this:

```
GITHUB_REPO="TheAICompanyLabs/mcp-proxy-open-source"
```

And update line 22 where it fetches the version to add the `-H "User-Agent: curl"` flag:

```
VERSION=$(curl -sL -H "User-Agent: curl" "https://api.github.com/repos/${GITHUB_REPO}/releases/latest" | g
```

Once `scripts/install.sh` is pushed with that repo name, run the one-liner again on your Mac:

```
curl -sSfL https://raw.githubusercontent.com/TheAICompanyLabs/mcp-proxy-open-source/main/scripts/install.s
```

Does `curl -sL ../install.sh | grep "GITHUB_REPO="` show the company repository name on your end?

User prompt: It ran the following install successfully but gave the following when running `mcp-proxy curl -sSfL`

```
https://raw.githubusercontent.com/TheAICompanyLabs/mcp-proxy-open-source/main/scripts/install.sh | bash 🚀 Installing Universal MCP
Proxy... -> Downloading v1.0.0 for darwin/arm64... -> Installing to /usr/local/bin (may require sudo password)... Password: ✅ Installation
complete! Run 'mcp-proxy' to get started. (base) avi@Avis-MacBook-Air ~ % mcp-proxy Verifying enterprise license with central server...
2026/09/03 01:56:33 ❌ Access Denied: Invalid or inactive license key
```

Response: The reason you are seeing that `Access Denied` error is actually a brilliant confirmation that your proxy is working exactly as designed—but your Mac is not a "clean room" just yet!

Here is exactly what happened: When you ran `mcp-proxy` with no arguments, the code executed `resolveLicenseKey()`. Because you used this exact same Mac to test the `handleLogin()` flow and the Enterprise Stripe keys earlier in the sprint, your Go proxy found that **old, saved configuration file** sitting locally on your hard drive (likely holding a test key).

Because the key wasn't empty, the proxy skipped the "Zero Key Boot" and immediately tried to verify that old cached key with your Next.js server, which correctly rejected it as inactive.

The Fix: Purge the Local Cache

To truly simulate a fresh developer downloading your tool for the first time, we must delete that cached configuration file from your Mac.

1. **Delete the Config File:** Depending on how `SaveConfig()` was written earlier in your project, the file is usually stored in your home directory. Run this in your terminal to wipe it:

```
rm -rf ~/.mcp-proxy
rm -rf ~/.config/mcp-proxy
```

(If your `SaveConfig` uses a different path, delete that specific file or folder).

2. **Verify Environment Variables:** Ensure you don't have the key exported in your current terminal session:

```
unset MCP_LICENSE_KEY
```

3. **Run the Clean Test Again:** Now that the machine is scrubbed of all Enterprise keys, launch the proxy again:

```
mcp-proxy npx -y @modelcontextprotocol/server-everything
```

You should now see the 🚀 `Booting Universal MCP Proxy in Free Open Core Mode...` message instantly appear, proving the air-gapped Free Tier works flawlessly for the public. Let me know when you see that zero-key boot screen!

User prompt: Okay this is running now lets test it on windows as well

Response: Excellent. With macOS completely verified, let's lock in the Windows environment.

Since you might have tested the proxy on this Windows machine earlier in the development cycle, we need to apply the exact same "Clean Room" treatment to ensure no old enterprise keys are hiding in the Windows file system.

Here is the exact PowerShell sequence to wipe the cache and run the Free Tier test:

Step 1: Open a Fresh PowerShell Window

Close your current PowerShell window and open a new one. This guarantees the system environment `PATH` update we applied during the installation script actually takes effect in the current session.

Step 2: Purge the Windows Cache

Copy and paste this block into PowerShell. It will silently delete any hidden configuration folders and unset any lingering environment variables:

```
Remove-Item -Recurse -Force "$env:USERPROFILE\.mcp-proxy" -ErrorAction SilentlyContinue
```



```
Remove-Item -Recurse -Force "$env:USERPROFILE\.config\mcp-proxy" -ErrorAction SilentlyContinue
Remove-Item -Recurse -Force "$env:APPDATA\mcp-proxy" -ErrorAction SilentlyContinue
$env:MCP_LICENSE_KEY = ""
```

Step 3: Boot the Proxy

Because we added `C:\mcp-proxy` to your system PATH earlier, you can now launch the proxy globally from any folder, exactly like on Mac. Run:

```
mcp-proxy npx -y @modelcontextprotocol/server-everything
```

(Note: If PowerShell says `mcp-proxy` is not recognized, it means the PATH variable hasn't refreshed yet. You can bypass it for this test by running the direct path: `C:\mcp-proxy\mcp-proxy.exe npx -y @modelcontextprotocol/server-everything`).

You should see the exact same 🚀 Booting Universal MCP Proxy in Free Open Core Mode... message pop up on Windows.

Once you confirm Windows is green-lit, the Open Source Free Tier deployment is officially and entirely complete. Are we ready to transition back to your private `mcp-web-dashboard` repository to wire up the live Stripe revenue integration?

User prompt: Perfectly loaded in windows as well!! Move towards the remaining items for making it a deployable and marketable product... do you have the list for list of marketing and developer guide and similar items listed for aggressive marketing

Response: RSVP: Launch marketing is a completely different muscle from engineering, but it requires the exact same structured discipline. You have a verified Open Core product that solves a massive security problem (preventing arbitrary code execution from AI agents). Now, we must weaponize that value proposition.

Here is your prioritized launch sequence to generate aggressive, high-quality developer traffic over the next 10 days.

1. The "Zero-Friction" Assets (Do this today)

Before you drive traffic anywhere, your destination must be optimized for conversion. Developers will bounce in 5 seconds if they do not immediately understand what the tool does and how to run it.

- **The 5-Second GIF:** Record a terminal screen showing an AI agent attempting a malicious command (e.g., `rm -rf /` or `DROP TABLE`), followed instantly by your TUI flashing red and blocking it via a `policy.yaml` rule. Put this at the very top of your GitHub `README.md`.
- **The Policy Cookbook:** Developers hate writing configurations from scratch. Create a `docs/cookbook.md` file in your repository with copy-paste YAML templates for popular tools (e.g., "Read-Only Postgres", "Sandbox Filesystem").
- **The Architecture Diagram:** Use Excalidraw to sketch a simple diagram showing the Agent -> MCP Proxy (Layer 1 & 2 Security) -> MCP Server flow. Developers need to visually grasp that your proxy sits directly in the middle as a physical circuit breaker.

2. High-Signal Launch Channels (The 48-Hour Blitz)

You must tailor your messaging to the specific technical culture of each platform.

- **Hacker News (Show HN):**
 - **The Hook:** Focus entirely on the technical problem. Title idea: "Show HN: An Open-Source Security Proxy for Model Context Protocol (MCP)".
 - **The Content:** Do not use marketing jargon. Explain *why* you built it: "I noticed giving Claude Desktop direct access to my filesystem MCP was a massive security risk. I built this Go proxy to intercept the JSON-RPC payloads in microseconds and enforce Layer 1/2 regex policies before they execute."
- **Reddit (r/LocalLLaMA, r/MachineLearning, r/ClaudeAI):**
 - **The Hook:** Focus on local safety and "Shadow AI."
 - **The Content:** Share a short tutorial on how to wrap standard MCP servers (like `@modelcontextprotocol/server-filesystem`) with your proxy to prevent catastrophic hallucinated commands.
- **MCP Ecosystem Registries:**
 - Submit your proxy to the **MCP Registry** and open-source lists like `awesome-mcp-servers`. While it is a proxy, not a standard server, the community desperately needs middleware security tools.

3. The Enterprise Upsell (Post-Launch Transition)

The goal of the Free Tier is volume. Once they install it via `curl` and use it locally, you introduce the Enterprise features naturally.

- **TUI Call-to-Action:** When a user boots the free proxy, ensure the console output says something like: [Tip: Need remote policy syncing and audit logs? Upgrade at `mcp-proxy-pink.vercel.app`].
- **Product Hunt (Launch 2.0):** Save Product Hunt for when your Enterprise Web Dashboard and Stripe billing are fully polished. Product Hunt audiences respond better to beautiful web interfaces and team-collaboration features than pure terminal utilities.

If we tackle the README assets first, would you like to draft the exact `README.md` structure or script out the 5-second terminal GIF?

User prompt: Not 10 days btw, we have to do this asap, we need to be aggressive remember, after which we will start working and polishing the

enterprise dashboards within 3 days of official launching free version, we need to start marketing for enterprise version as soon as dashboard is done. When marketing and making plan keep this as a note

Response: Understood. Let's apply a strict "Bias for Action" to this timeline, leveraging the operational rigor from your time at Amazon to ship this immediately. We are shifting from a 10-day rollout to a hyper-aggressive 72-hour execution plan.

Here is your condensed, high-velocity blueprint to launch the Free version today, polish the Enterprise dashboard over the next 72 hours, and pivot directly into B2B marketing.

Phase 1: The 24-Hour Open-Source Blitz (Execution: TODAY)

The immediate goal is pure volume: get developers to run the `curl` command and star the repository.

- **Asset Polish (Next 2 Hours):** Record the 5-second terminal GIF. Since your terminal interface is constructed using Bubble Tea and Lip Gloss, ensure the visual feedback for a blocked malicious command is vibrant and unmistakable in the recording.
- **The Hard-Hitting Copy:** Do not be vague. Explicitly list your core capabilities in the README and launch posts: highlight the dynamic policy loading, runtime guardrails, SSRF protection, directory traversal blocklists, and prompt injection detection.
- **Aggressive Distribution:** Drop the "Show HN" post, publish to [r/LocalLLaMA](#) and [r/MachineLearning](#), and submit the repository to every Model Context Protocol Discord and registry you can find.

Phase 2: The 3-Day Enterprise Polish (Execution: Days 2-4)

While the community tests the Free tier, we ruthlessly lock down the commercial infrastructure in your private repository.

- **Dashboard Finalization:** Complete the remaining UI/UX routes for the web dashboard using Next.js 14, Vercel, Neon PostgreSQL, and Prisma ORM.
- **Revenue Pipeline:** Switch your Stripe integration to live mode and rigorously test the Stripe billing infrastructure and webhook logic to ensure zero-touch license generation.
- **Access Control:** Perform a final audit on the NextAuth implementation with GitHub OAuth to ensure enterprise user sessions are strictly isolated.

Phase 3: The Enterprise Go-To-Market (Execution: Day 4+)

The moment the dashboard is deployed, the marketing message shifts entirely from "local developer tool" to "team-wide B2B security."

- **The Conversion Funnel:** Add a clear call-to-action in the Free version's terminal output directing users to the web dashboard for team features.
- **Direct Developer Outreach:** Monitor GitHub for developers who star the repository or open issues. Reach out to them directly on LinkedIn or Twitter: "Glad you like the local proxy. We just launched the Enterprise dashboard for remote policy syncing and audit logs—want a trial key for your team?"
- **Content Marketing:** Publish a technical deep-dive on how your architecture intercepts tool calls in microseconds.

To get the Free version out the door today, which piece of the GitHub README do you want to knock out right now: the architecture diagram, the copy-paste YAML policies, or the security feature list?

User prompt: We have completed phase 1 today, in which we have completed the following listed below, 1. Phase 1 Completion Summary We have successfully executed Phase 1: Open-Source Go Proxy Launch. ◦ Extracted the Core: Separated the Go proxy from the Next.js dashboard into an independent repository (mcp-proxy-open-source). ◦ Built the Security Engine: Engineered a zero-latency stdio interceptor that parses JSON-RPC payloads and evaluates regex conditions in microseconds. ◦ Validated the Sandbox: Successfully tested local fail-closed behavior, proving the proxy instantly crashes and locks down if the policy file is corrupted or missing. ◦ Live Threat Interception: Proved the Go engine successfully intercepts and blocks live Directory Traversal (../), SSRF (169.254...), and Destructive SQL (DROP TABLE) attacks. ◦ Release Pipelines: Established cross-platform binaries (macOS, Linux, Windows) with one-line install scripts (curl and PowerShell). ◦ The Policy Cookbook: Authored comprehensive YAML templates for the top 50 MCP servers across 6 critical threat vectors. ◦ Developer Experience (DX): Finalized an interactive, 60-second Quickstart guide and beginner-friendly policy setup flow for the README.md. ◦ Launched: Deployed to Hacker News and relevant subreddits. 2. Below is the Master Roadmap (Phases 1-4) detail view in your previous response, 1. Phase 1: Open-Source Go Proxy Launch (COMPLETED) ◦ Deliver the air-gapped, zero-latency local firewall. ◦ Establish the baseline developer community via Hacker News. 2. Phase 2: The SaaS Dashboard (Stripe & Auth) ◦ Transition to the mcp-proxy-dashboard repository. ◦ Integrate Stripe billing for the Enterprise tier. ◦ Implement user authentication and API key generation (MCP_LICENSE_KEY). 3. Phase 3: Centralized Policy & Telemetry (The Enterprise Moat) ◦ Connect the local Go proxy to the Next.js backend. ◦ Enable remote pulling of centralized policies. ◦ Implement cryptographic hash-chained audit logs streaming from the proxy to the dashboard. 4. Phase 4: SIEM Integration & Compliance ◦ Build data pipelines routing audit logs to Datadog, Splunk, and OpenSearch. ◦ Finalize SOC 2 / HIPAA compliance positioning. ◦ Implement dynamic IAM Role federation (AssumeRole). 3. Attaching the context summary below, reconfirm and recheck with previous context, after which we will begin phase 2 in this chat, Context Payload: Transitioning from Phase 1 to Phase 2 Project State We have successfully launched the open-source Universal MCP Proxy (Phase 1). It is a zero-latency Go middleware that intercepts MCP stdio traffic, enforcing strict YAML-based regex policies to prevent SSRF, directory traversal, and destructive executions. It operates completely air-gapped on the local machine with a fail-closed architecture. Technical Milestones Reached ◦ The Go engine (policy.go) accurately unmarshals policy.yaml using a specific struct (policies: array, argument:, operator: "regex", value:). ◦ Extensive local testing verified that the Go regex engine successfully intercepts live JSON-RPC payloads (e.g., blocking ../etc/passwd on read_file). ◦ Installer scripts (curl for Unix, .ps1 for Windows) are live and downloading the compiled binaries. ◦ The repository includes a cookbook/ directory with pre-built profiles for different developer personas (e.g., "Read-Only Analyst"). Strategic Direction (The Enterprise Moat) We established a strict boundary between the free tier and the Enterprise V2 tier. The free version provides ephemeral, local terminal logging. The enterprise moat relies on tamper-evident, hash-chained audit logs, dynamic IAM isolation (AWS AssumeRole), centralized policy registries, and SIEM integration. We do not give away centralized telemetry for free. Next Immediate Steps (Phase 2 Start) The next session will focus entirely on Phase 2: The Next.js SaaS Dashboard. We will move away from the Go proxy repository and open the dashboard codebase to integrate Stripe billing, user authentication, and the generation of the MCP_LICENSE_KEY required to connect the proxy to the cloud backend.

Response: The context is locked and reconfirmed. The decoupling is complete: the public `mcp-proxy-open-source` repository is successfully handling local, air-gapped threat interception, while the proprietary enterprise moat remains securely isolated.

With Phase 1 shipped, the 72-hour clock for the Enterprise Go-To-Market starts now.

We are shifting entirely to the private `mcp-web-dashboard` repository. To execute Phase 2 rapidly and handle high concurrency during the enterprise launch, we will leverage Next.js 14 deployed on Vercel, backed by a Neon PostgreSQL database for efficient edge connection pooling, and managed via Prisma ORM. Access will be strictly gated using NextAuth with GitHub OAuth, leading directly into the Stripe billing infrastructure.

Here is the immediate technical sprint to build the SaaS Dashboard and the `MCP_LICENSE_KEY` generation engine.

Step 1: The Prisma Schema (Neon PostgreSQL)

Before wiring up authentication or billing, the database schema must be established to handle the relationship between users, their subscriptions, and their license keys. We need to define models for `User`, `Account` (for NextAuth GitHub OAuth), and a `License` table that stores the generated `mcp_live_...` key, ties it to a specific Stripe Subscription ID, and tracks its `status` (ACTIVE, REVOKED, EXPIRED).

Step 2: NextAuth & GitHub OAuth Integration

Configure Next.js to handle user sessions. When an enterprise developer logs in, they authenticate via GitHub. This provisions their `User` record in the Neon database and drops them into the protected dashboard route where they can manage their billing and view their active keys.

Step 3: Stripe Checkout & Webhook Infrastructure

This is the core revenue engine.

- **The Frontend:** A Next.js route that redirects the authenticated user to a Stripe Checkout Session for the Enterprise Tier.
- **The Webhook (`/api/webhooks/stripe`):** A critical backend endpoint that listens for the `checkout.session.completed` event from Stripe. Upon receipt, this function automatically generates a cryptographically secure `mcp_live_...` key, associates it with the user's Prisma record, and activates it.

Step 4: The Verification Endpoint Activation

We will finalize the `/api/license/verify` endpoint we drafted earlier with the in-memory rate limiter. It will now actively query the Neon database via Prisma to ensure the key passed from the local Go proxy is valid, active, and tied to a paying Stripe subscription.

To get the dashboard infrastructure standing immediately, where should we begin the code implementation: defining the **Prisma Database Schema**, setting up the **NextAuth GitHub OAuth flow**, or constructing the **Stripe Webhook handler**?

User prompt: Following is the summary of the current state of development related to dashboard: Completed Tasks (Infrastructure & API) ◦ Core Stack Provisioning: The Next.js 14 application is fully scaffolded and deployed on Vercel, connected to your Neon PostgreSQL serverless database via Prisma ORM. ◦ Authentication Pipeline: NextAuth is integrated and configured with GitHub OAuth, successfully capturing developer identities. ◦ The Verification Engine: The `/api/license/verify` POST route is fully operational. It accurately queries the Neon database, validates key statuses (ACTIVE), and returns the exact JSON contract (tier, features) required by the Go CLI. ◦ Endpoint Security: A custom, in-memory sliding-window rate limiter is deployed on the verification route, successfully rejecting brute-force attacks (429 Too Many Requests) to protect your database connection pool. ◦ Payment & Sync Logic (Test Mode): The baseline Stripe subscription infrastructure and webhook handlers are written and successfully tested using Stripe's mock environment. The database migrations for license tracking are applied. ⌚ Remaining Tasks (The 3-Day Production Sprint) ◦ Stripe Live Cutover: We must flip Stripe from Test to Live mode, generate production API keys/webhook secrets, and securely inject them into your Vercel environment variables. ◦ Legal Assets: Stripe Live and GitHub OAuth strictly require public-facing legal pages. We need to rapidly deploy a standard Privacy Policy and Terms of Service to the website. ◦ UI/UX Polish: The frontend dashboard routes need to be finalized so developers have a beautiful, intuitive interface to copy their `mcp_live_...` keys, view their active tier, and access the documentation. ◦ Prisma Connection Pooling: We need to explicitly configure Prisma to route through Neon's connection pooler. Because the CLI proxy queries the database on every startup, we must ensure Vercel serverless functions don't exhaust your database connections during a traffic spike. ◦ The "Real Money" End-to-End Test: Execute one complete, live checkout simulation. You will log in via GitHub, process a live credit card payment, verify the Stripe webhook successfully updates the Neon database, and boot the proxy using the newly minted key.

Response: With the backend architecture validated and the test environment green-lit, we are in the final 72-hour window. The focus shifts entirely to production hardening and revenue activation.

If we go live with Stripe and execute the marketing blitz without connection pooling, the resulting traffic spike will exhaust your database connections. Every time a new developer runs the CLI, the Vercel serverless function spins up, opens a database connection, verifies the key, and closes it. Without a pooler, Neon will max out its connection limits and start dropping requests, breaking the proxy for paying users.

Here is how we knock out the two immediate blockers to prepare for the live Stripe cutover:

1. Infrastructure Hardening: Prisma Connection Pooling (Neon)

Neon natively supports PgBouncer for serverless connection pooling. We must separate your database URL into a pooled connection (for the Next.js API routes) and a direct connection (for Prisma migrations).

Update your `schema.prisma`:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")
}
```

Update your Vercel Environment Variables: You will need to fetch these from your Neon dashboard under **Connection Details**:

- `DATABASE_URL`: Set this to your **Pooled connection** string (Neon adds `-pooler` to the endpoint URL or requires `?pgbouncer=true`).
- `DIRECT_URL`: Set this to your standard, non-pooled connection string.

2. Compliance Blockers: Legal Assets

Stripe will not allow you to activate Live Mode, and GitHub OAuth will flag your production app, without a public Privacy Policy and Terms of Service. We do not need a lawyer for this phase; we need standard boilerplate pages deployed immediately.

Create two static Next.js routes:

1. `app/privacy/page.tsx`
2. `app/terms/page.tsx`

These pages simply need to state that TheAICompanyLabs collects email addresses for account management, uses Stripe for payment processing, and does not sell developer data.

Next Step

Which of these two critical path items should we execute right now?

1. I can provide the exact Next.js page code for the **Legal Assets** so you can deploy them and unlock Stripe Live today.
 2. We can walk through the **Neon Dashboard** to grab the exact connection strings and lock down the Prisma pooling.
-